Java Container Awareness

Version 1.0.0, 2026-10-15

Home

Introduction

Course Title: Java Container Awareness

Course description: This course provides a practical introduction to running Java applications as a fully containerized application, with a focus on resource detection and memory management inside a container. Learn how containerized Java applications integrate with Kubernetes and OpenShift and how to identify common runtime and resource-related issues. Explore essential troubleshooting techniques and tools for containerized Java workloads, particularly for Red Hat build of OpenJDK images, but it can be generalized for non-OCP 4.

Duration: 2 Hours

Topics covered in this course

- Introduction to Containerized Java
- Container Resource Detection
- Risks and Memory Management
- OpenShift and Kubernetes Integration
- Troubleshooting and Tooling
- Hands-on Lab

Prerequisites

This course assumes that you have the following prior experience:

- Experience with Java (in general) and heap (off-heap) allocation
- Experience with container runtimes/engines such as podman or Docker
- Experience with container orchestrators, such as Openshift in terms of deployment (Operators knowledge is not required and just build on top of deployments)

Use of Artificial Intelligence: This publication contains content or media that is assisted by AI tools generation. The Training deliverables have primarily human specialist authorship, and all content was reviewed by an human domain experts.

Lab Environment

This lab requires a standard OpenShift Container Platform (OCP) environment with permissions to deploy an application using a StatefulSet, Pod YAML, or Deployment. You may use any OpenShift cluster you have access to. If you do not have access to an OpenShift cluster for the hands-on exercise, follow the instructions on this page to order a lab environment.

Instructions to Launch Your Lab on the Red Hat Demo Platform (RHDP)

1. **Log in** to the RHDP portal
2. **Search** for the catalog entry: **Red Hat OpenShift Container Platform Cluster (Multi-Cloud)**.
3. **Click** on the catalog entry in the search results.
4. On the catalog page, **click** the **Order** button.
5. **Fill out** the required details in the order form.
    - **Activity:** *Practice / Enablement*
    - **Purpose:** *Learning about the product*
    - **Cloud Provider:** *cnv*
    - **OpenShift Version:** *4.21 (or as desired)*
    - **cluster size:** *sno*
6. **Review the warning** at the bottom of the form and **check the box** labeled:
   “I confirm that I understand the above warnings.”
7. **Click** the **Order** button to place your lab order.

Important Notes:

- This lab may take approximately **5 minutes** to become ready.
- You will receive an **email with access details** once your lab environment is ready.
- You can also **retrieve lab access** directly from the RHDP portal.

How to Access Your Lab via the RHDP Portal:

1. On the RHDP portal, **click** on the **Services** option in the left-hand menu.
2. **Select your lab** from the listings on the right-hand side of the page to view access details.

RHDP Portal Links

- RedHat associates: https://demo.redhat.com/
- RedHat partners: https://partner.demo.redhat.com/

Introduction to Containerized Java

Containerized Java focuses on the evolution of the Java Virtual Machine (JVM) to operate bounded and efficiently within container environments. Therefore this will impact deployments in orchestrors such as OpenShift and Kubernetes. Currently, Java runtimes utilize container awareness to detect cgroup limits, allowing the JVM to automatically adjust its resource restriction based on container boundaries rather than host hardware limits.

Key aspects include:

- **Evolution of Java:** Shifting from host-based resource perception to environment-aware detection.
- **Container Awareness:** The ability of the JVM to recognize and respect memory and CPU limits imposed by the container runtime.
- **Cgroup Detection:** Leveraging kernel-level control groups to monitor and manage resource availability accurately.
- **Operational Efficiency:** Simplifying deployments by ensuring the JVM aligns its internal memory and thread management with defined container resource limits.
- **Red Hat build of OpenJDK:** Red Hat build of OpenJDK containers, have specific settings defined in the run-java.sh script, which is the default `ENTRYPOINT` when using our image and adds features and configurations.

The images of Red Hat build of OpenJDK are fully supported on Openshift 4 and podman. Additionally they are also tested on podman-docker emulator, therefore the images are fully expected to work on Docker including the s2i Toolkit.

Inside a container, Java ignores CPU requests and only takes into consideration CPU limits. Finally, it behaves exactly the same in a container as in off-container, the only difference is the memory and CPU limits used where a container will be usually a subset of the host, unless unbouded deployment is used.

This course is fully based on the current OpenJDK implementation and the article How to use Java container awareness in OpenShift 4.

Evolution of Java in containerized environments

The landscape of Java application deployment has shifted dramatically with the advent of containerization. Initially designed for virtual machines and bare-metal hardware, the Java Virtual Machine (JVM) faced significant architectural hurdles when introduced to the world of Linux containers.

Non-container-aware Java

Before 2019, Java (its JVM) would identify the system resources by querying the underlying operating system (OS) kernel via `/proc/meminfo` and `/proc/stat`. When a Java process runs on a physical server, it sees the total memory and CPU cores of the host.

In other words, despite running inside a container, Java was unable to detect the container's limits. It would only detect the host limits regardless of the deployment. Consequently, the DevOps team needed to set the heap limits (Xmx) manually as a size and make sure those two were approapriate: container size and Java heap size.

Partial container awareness

Later, evolving this concept, Java was it was encapsulated via an initialization script: run-java.sh, which would then detect the container limits and deploy Java with those limits via Xmx (script → detect container limits → start Java). Java did not know about cgroups itself. It would just start with the values provided by the initialization script (e.g. Xmx). The initization script would usually set only heap values and not isolate the CPU settings for Java.

JDK-8146115

The evolution toward true container awareness reached a milestone with **JDK8146115**. This critical enhancement, backported to Java 8 (starting with 8u191) and native to all newer versions (Java 11, 17, 21+), fundamentally changed how the JVM initializes.

Instead of looking at the host's total resources or needing a hard-coded value for `Xmx`, the JVM is able to fetch the cgroup hierarchy directly. This allows the JVM to:

1. **Read Container Memory Limits:** Automatically calculate maximum heap size as a percentage based on the cgroups memory limits of the container. Not request.
2. **Detect CPU Availability:** Properly identify the number of CPU shares or quotas allocated to the container, allowing the garbage collector and internal thread pools to scale their concurrency appropriately.

Containers achieve isolation through **Control Groups (cgroups)**, which is a Linux kernel feature that limits, accounts for, and isolates the resource usage of a collection of processes. Before 2019, Java was not cgroup-aware when running inside a container. This would create a mismatch where the JVM believed it had access to gigabytes of host RAM, leading to:

- **OOMKills:** The JVM would attempt to allocate heap memory based on host capacity, triggering the container orchestrator to kill the process for exceeding its cgroup memory limit.
- **Performance Degradation:** The JVM would size its internal thread pools (e.g., ForkJoinPool) based on the total physical CPU cores of the host, leading to excessive context switching and performance bottlenecks in environments where the container was throttled, given the container likely has different CPU limits.

Current Fully Container-Aware Java

Currently, container awareness is the standard expectation for cgroups v1 and cgroups v2 complaince Java development for Java 8u372+, 11.0.16+, 17+ such as 21 and 25. The transition from unaware to fully container-aware has considerably streamlined the DevOps workflow:

- **Self-Tuning:** Applications can now adjust their memory footprint automatically to fit within Kubernetes/OpenShift resource requests and limits upon start/restart.

- **Improved Stability:** By aligning the JVM's view of resources with the kernel-enforced limits, the frequency of container restarts due to resource exhaustion is drastically reduced.
- **Reduced Configuration Overhead:** Developers do not spend time tuning CPU values and heap limits (`-Xmx`), as the JVM can now effectively derive them from the container environment deployed by an orchestrator via percentage automatically by having MaxRAMPercentage at 25% (upstream value).

Specifically in terms of Red Hat build of OpenJDK, Red hat's OpenJDK images rely in a custom script `run-java.sh` script as `ENTRYPOINT`, which currently is not use anymore for detecting limits from the container but to add features and optimizations: such as ParallelGc, `MaxRAMPercentage` at 80%, `JAVA_OPTS` and `JAVA_OPTS_APPEND` environment variables. The current 80% heap limit therefore does not come by default, but rather by the script's `MaxRAMPercentage`.

Understanding this evolution is important for any team moving toward microservices architectures. Container awareness provides a foundation that requires the Development team (who handles the Java settings) and the Deployment team (who handles the container size) to communicate and understand that Java will behave differently at runtime inside a container according to its runtime settings to avoid diffent behaviour.

This difference in behaviours at runtime can be presented as JVM heap memory, thread difference, GC collector differences, and off-heap memory differences given different container's limits. Therefore, this is a topic that remains a pivotal consideration for production-grade containerized deployments.

Example deployment

Below is an application that is deployed with Red Hat's build of OpenJDK using the default ENTRYPOINT (run-java.sh):

```
FROM registry.access.redhat.com/ubi9/openjdk-17:latest

# Copy the compiled JAR into the standard deployments folder
COPY target/hotrod-app.jar /deployments/hotrod-app.jar
```

Overview of container-aware versus non-container-aware Java

Before container awareness, Java applications more often than not would struggle to coexist within its container limits demanding direct intervention from DevOps teams. In summary, the Java Virtual Machine (JVM) was designed to use hard-coded values from the host operating system for resource availability, while containers rely on control groups (cgroups) to impose different resource constraints.

The Evolution of Container Awareness

Before the introduction of container awareness, the JVM would query the host machine's total physical memory and CPU cores, completely ignoring the limits imposed by the container runtime.

Feature	Non-Container-Aware	Fully Container-Aware
Memory Detection	Sees total host RAM	Sees container (cgroup) defined memory limit
CPU Detection	Sees total host CPU cores	Sees container (cgroup) defined CPU limit
Heap Sizing	Based on total host memory	Based on container memory limit
OOM Risk	High (dissociated with the container size)	Lower (associated with the contianer size)

Container awareness is the default (already enabled) and the user needs to disable it via manually setting the flag `-UseContainerSupport` when starting Java:

State	JVM Flag	Behavior
Default (Enabled)	`-XX:+UseContainerSupport` (Implicit)	Container awareness is enabled by default. If you do not set any flag, OpenJDK automatically detects container limits.
Disabled	`-XX:-UseContainerSupport`	Container awareness is explicitly disabled. The user must manually pass this flag when starting Java to turn the feature off.

Non-Container-Aware Java

In a non-container-aware Java deployment, if you deploy a Java application with a heap size into a container with a 1Gi limit, the JVM's Xmx may be significantly larger than the 1Gi container, causing immediate issues.

When the JVM allocates this memory, the host kernel will trigger a cgroup **Out-of-Memory (OOM) Kill** event. The process is terminated abruptly because it breached the container's allowed boundary, despite the JVM assuming it had plenty of memory available. Note that JVM allocation and application consumption are separated and may considerably diverge, because the JVM may allocate more than the application is in fact currently using.

Partial container-awareness - transition period

Before detecting the values nativerly, Red Hat OpenJDK passed through a periodic where the heap values would be calculated from an wrapper script, i.e. an initilization script, which would detect the cgroups limits and then start Java (usually via a value for `Xmx`).

This initialization scripts are still present in some Red Hat Java container images, however not for cgroups detection anymore, instead only for JVM settings such as Garbage Collection and JVM parameters.

Fully Container-Aware Java

The implementation shift occurred with the implementation of **JDK-8146115**, starting with Java 8u191 implementation and then improved with other jiras, currently Java relies on cgroups natively for detection and memory percentage bootstrap. In other words, those updates enabled the JVM to detect its container `cgroup` filesystem natively, rather than relying on a initialization script for limit detection. The JVM natively starts Java heap as a percentage of the limits detected and uses the CPU limits for thread calculations.

When running a container-aware JVM, the process performs the following detection:

- **Memory Detection:** Instead of asking the OS for total physical RAM, the JVM reads the cgroups (cgroups v1: `memory.limit_in_bytes` and cgroups v2: `/sys/fs/cgroup/memory.max`) file from the container's cgroup boundaries. The JVM then calculates the default heap size as a percentage of this limit, rather than a percentage of the host's total RAM or a hard-coded value, automatically.
- **CPU Detection:** The JVM queries the cgroups CPU files to determine how many CPU cores are available to the container. And calculates the thread pools from this value, rather than the host limits. Example:

Table 1. Java cgroups v2 example detection

Command	Raw Output	Interpreted Limit
`$ cat /sys/fs/cgroup/cpu.max`	`200000 100000`	**2 CPUs is the limit** (Quota: 200,000 / Period: 100,000) Java ignores requests.
`$ cat /sys/fs/cgroup/memory.max`	`2147483648`	**2 GiB** (2,147,483,648 bytes)

Why This Matters for current Java Deployments

Transitioning to container-aware Java is not merely a technical optimization; it is a prerequisite for stability in containers and easy deploying inside orchestrators like OpenShift and Kubernetes.

- **Simplified Configuration:** Developers no longer need to hardcode `-Xmx` values that perfectly match every application and environment resource constraints. The JVM automatically detects the `limits` defined from the container/cgroups limits from the deployment manifests and uses percentage as reference adapting for a new value.
- **Deterministic Behavior:** By aligning the JVM's view of resources with the container's reality, you reduce the likelihood of misbehaviors such as OOM Kills that occur when the JVM allocates memory, given the JVM won't over-allocate (above the container size).
- **Operational Efficiency:** Container-aware Java allows DevOps teams to set resource requests and limits in OpenShift with the confidence that the application will scale its internal memory management to fit the provided space.

The default value of `MaxRAMPercentage` is 25% for upstream deployment (straight java command inside a container), however Red Hat build of OpenJDK takes advantage of `run-java.sh` (as container `ENTRYPOINT`) to add the following features:

- The heap will be calculated at 50% of the container size (not the host). Later images will default to 80%.

- The user can use `JAVA_OPTS` and other environment variables from run-java.sh, such as `JAVA_OPTS`, `JAVA_OPTS_APPEND`, `GC_MAX_METASPACE_SIZE` for runtime tuning.
- The user will have ParallelGC as default GC collector and other JVM settings (which can be overwritten with `JAVA_OPTS`).

The specific arguments for each image can be found: ubi9/openjdk-21

> ❗ Although Java container awareness automatically calculates heap size and thread pools based on container limits memory and CPU respectively, it does not manage off-heap memory limit, which becomes the remainder of the memory avaliable in the container. Off-heap components include metaspace, code cache, direct buffers, and native libraries fall outside the scope of `MaxRAMPercentage`, consequently. In other words `MaxRAMPercentage`only accounts the heap percentage. Therefore, you must thoroughly benchmark your application to balance heap and off-heap allocations to prevent container OOMKill events.

Importance of cgroup detection for deployments

In cloud-native architectures, the shift from virtual machines to containerized runtimes has fundamentally changed how applications interact with hardware. For Java, historically accustomed to seeing the full host OS settings, the introduction of cgroups (control groups) represents a change and evolution in its resource detection and management.

The Evolution of Java and Resource Visibility

Before container-aware JVMs, a Java application running inside a container would often query the underlying host OS to determine available memory and CPU. If the container was restricted to 2GB of memory, but the host node had 64GB, the JVM would frequently miscalculate its maximum heap size (`-Xmx`). This led to two primary failure modes:

1. **OOMKiller Invocation:** The JVM, believing it had access to a larger pool of memory, would allocate heap space that exceeded the container's hard limit. The Linux kernel, seeing the container exceed its cgroup limit, would trigger an Out-of-Memory (OOM) Kill, abruptly terminating the process.
2. **Resource Inefficiency:** Conversely, if the JVM underestimated the capacity, it might fail to utilize the resources explicitly allocated to it by the orchestrator, leading to poor performance and degraded throughput.

Why cgroup Detection is Critical

Cgroup detection is the mechanism by which the JVM detects the boundaries imposed by the container runtime. Its importance for deployments cannot be overstated for the following reasons:

- **Boundary Compliance:** It ensures the JVM respects the container's quota rather than the physical host's capacit by reading from cgroups hierarch (`/sys/fs/cgroup/`) on both cgroups v1 and cgroups v2. The JVM adjusts its heap and CPU settings (for thread calculation).
- **Preventing Failures:** With proper cgroup detection, the JVM aligns its garbage collection

triggers and heap allocation thresholds with the container's memory limit. This prevents the scenario where the application performs perfectly under light load but crashes instantly when the memory pressure when crossed the cgroup ceiling.

- **Orchestration Alignment:** Orchestrators like OpenShift and Kubernetes allow deployment of containers that have Resource Limits and/or Requests and those are enforced by the Kubelet and the kernel. Although Java ignores requests, Java's ability to read the CPU and memory limits allows the JVM to operate nominally inside a container regardless of the host. Meaning, given JVM is container-aware, it can effectively use flags like `-XX:MaxRAMPercentage`, ensuring the heap scales directly with the container's assigned limit when it is deployed. Users can tune those percentage values.

Implications for DevOps teams

For a DevOps team, relying on container-aware Java simplifies the deployment pipeline. You no longer need to hardcode `-Xmx` values for every environment. By allowing the JVM to detect the cgroup limit, you can:

- **Standardize Images:** Use the same container image across development, testing, and production environments, where resource limits may vary significantly.
- **Predictable Scaling:** As you adjust memory limits in your Kubernetes manifest, the application automatically adapts its memory footprint without requiring a rebuild or manual configuration update.

If before the users set Xmx and the container size, currently they just need to know the adequate percentage and the heap will adapt to the container size automatically at start up.

Red Hat build of OpenJDK containers

Specifically the default OpenJDK container image, the default `ENTRYPOINT` will be run-java.sh, which is a script that provides default settings, such as ParallelGC as expected GC, but also environment virables that can be used at runtime:

Environment Variable	Purpose
`JAVA_OPTS`	Main option, which includes default settings - can be used to replace them
`JAVA_OPTS_APPEND`	Append option, can be used to complement the main ones
`GC_CONTAINER_OPTIONS`	Used for GC collector to be used.
`GC_MAX_METASPACE_SIZE`	The maximum metaspace size.
`GC_METASPACE_SIZE`	The initial metaspace size.

Although the ENTRYPOINT script adds those environment variables above, Java (natively by default) already provides other such as: `JAVA_TOOL_OPTIONS` and `_JAVA_OPTIONS`. Meaning, those are not inherited from the run-java.sh. However those extra environment variables are usually not recommended because they are used for any Java process inside the container, because they are detected by any Java process inside that container.

Example: Why JAVA_TOOL_OPTIONS should be usually avoided

```
$ export JAVA_TOOL_OPTIONS="EXAMPLE"
$ openjdk/app-injection$ jcmd
Picked up JAVA_TOOL_OPTIONS: EXAMPLE
Unrecognized option: EXAMPLE
Error: Could not create the Java Virtual Machine.
Error: A fatal exception has occurred. Program will exit.
```

That is to say that if you set `JAVA_TOOL_OPTIONS`, any Java process will pick it up: jcmd, jps, jstack for instance are all Java processes and will pick those variables.

For a deep dive into how these changes specifically affect Java runtimes specifically for cgroups v2, review the scenarios outlined in the following developer article:

- How does cgroups v2 impact Java, .NET, and Node.js in OpenShift 4? - Scenarios with Java

Key Takeaways for Java Developers

- OpenJDK 8u372+, 11.0.16+, 17, and 21 natively support cgroups v2 detection.
- Red Hat container images now rely on native OpenJDK C++ container detection rather than legacy `run-java.sh` wrapper scripts for setting memory limits.
- The `MaxRAMPercentage` property (defaulting to 80% in current images) allows the JVM to dynamically size its heap based on the cgroups v2 boundaries.

Additional References

For detailed information on the UBI 9 OpenJDK 21 container image, including available environment variables and configuration options, see the official container details:

- Red Hat OpenJDK 21 UBI 9 Container Image Documentation

> ❗ While cgroup detection prevents the JVM from over-allocating heap space by setting the Heap Percentage automatically and allowing the user to tune it, it does **not** manage off-heap memory (Metaspace, Code Cache, Thread stacks, etc.). Because the heap is only a portion of the total memory usage, administrators must still account for the non-heap overhead seperately. Failing to leave enough room for off-heap memory can still result in a cgroup OOMKill, even if the heap appears to be within the allowed percentage.

By ensuring your deployments utilize container-aware Java versions (such as Java 8u191+ or any later LTS version), you bridge the gap between the application’s runtime needs and the platform’s enforcement mechanisms, leading to more resilient and stable microservices. The user only needs to worry about the exact percentage to deploy the application.

Container Resource Detection

Container resource detection enables the Java Virtual Machine (JVM) to accurately identify resource constraints imposed by the container environment rather than the underlying host node.

- **JVM Evolution:** Starting with Java 8u191 (via JDK8146115), the JVM gained the ability to parse control group (cgroup) files to determine resource boundaries.
- **Memory Limit Detection:** The JVM identifies the specific memory limit assigned to the container, allowing it to scale the heap size appropriately based on available container resources.
- **CPU Availability:** The JVM detects the number of CPU cores allocated to the container, preventing over-provisioning and ensuring efficient thread management within the containerized environment.
- **Operational Benefits:** Accurate detection prevents OutOfMemory and OutOfMemoryKill caused by the JVM incorrectly defaulting to host-level physical memory rather than container-specific limits.

In simple terms: do not not worry about the Xmx directly vs size of the container. Instead, focus on the adequate correct percentage value for heap inside a container and how the application behaves.

Understanding JDK8146115 and Java 8u191 Updates

Before the introduction of specific container-aware updates, the Java Virtual Machine (JVM) lacked the ability to natively read the resource boundaries imposed by container runtimes. This caused significant operational friction in cloud-native environments.

The Challenge: JVM Resource Blindness

In early containerized environments, the JVM would query the underlying host operating system for available resources (such as total system memory and available CPU cores) rather than the specific cgroup limits applied to the container.

This behavior led to a critical disconnect:

- **Memory Miscalculation:** The JVM would attempt to size its heap based a specific Xmx value, often exceeding the container’s hard memory limit set by Kubernetes or OpenShift. This would lead to OOM Killer occurences: once the Java process exceeded the container’s cgroup memory limit, the OCP node kernel’s Out-Of-Memory (OOM) Killer would terminate the process, leading to unexpected pod restarts.
- **CPU and thread calculation:** The CPUs limits would be detected from the host in any situation, leading to inefficient thread pool sizing and potential overhead, such as context-switching and multiple threads competing for resources and even including CPU throttling.

The Turning Point: JDK8146115

To address these issues, the OpenJDK community introduced **JDK8146115**. This enhancement provided the necessary infrastructure for the JVM to correctly read and respect cgroup memory and CPU limits.

Instead of querying `/proc/meminfo` or `/proc/stat` that reflect host-level resources, the JVM was updated to inspect the cgroup filesystem hierarch directly (`/sys/fs/cgroup/`). This allows the JVM to identify the specific constraints applied to the process namespace.

Java 8u191: Beginning of container-awareness

The release of **Java 8u191** that had JDK-8146115 provided the first step towards fully container awareness. This update backported the container-awareness features, bringing container detection capabilities to Java 8 deployments. After this version more improvements were done to have a full percentage setting of the heap based on the container size.

- **Native Cgroup Awareness:** The JVM automatically detects container limits at startup for both cgroups v1 and cgroups v2 (8u372, 11.0.16+, 17+).
- **Native Cgroup cgroups v2 compliance:** Both cgroups v1 and cgroups v2 (8u372, 11.0.16+, 17+) limits are detected.
- **Default Ergonomics:** The default heap size calculation logic was updated to use a fraction of the container's memory limit instead of the host's physical memory.
- **Specific JVM Flags:** Introduction of flags such as `-XX:MaxRAMPercentage`, `-XX:MinRAMPercentage`, and `-XX:InitialRAMPercentage`, allowing developers to define heap sizes as a ratio of the container limit rather than hard-coded values.

Impact on Current Deployments

With Java 8u191+ (and later versions), the relationship between the container (cgroups) and the JVM became **coupled**.

Feature	Legacy Behavior (Pre-8u191)	Current Behavior (After 8u191+)
Memory Source	Queries `/proc/meminfo` (Host RAM)	Queries cgroups (Container Limit)
Heap Sizing	Requires manual `-Xmx` configuration	Automatically scales based on `-XX:MaxRAMPercentage`
Operational Impact	Decoupled; changing container size requires manual JVM tuning	Coupled; scaling container limits automatically adjusts heap

By aligning the JVM's view of resources with the container orchestration layer, these updates eliminated the decoupling problem (container not matching the Java's heap size). Developers no longer need to manually synchronize `-Xmx` settings with pod resource limits, significantly simplifying the management of Java applications in Red Hat OpenShift and Kubernetes.

Detecting Container Memory Limits

In modern Java deployments, it is critical that the Java Virtual Machine (JVM) correctly identifies the memory boundaries imposed by the container orchestration platform. Failure to do so leads to performance degradation, inefficient resource utilization, or unexpected pod terminations.

Why Container Awareness Matters

Prior to the integration of `JDK8146115` (introduced in Java 8u191), the JVM relied on standard OS-level utilities to detect available memory. In a containerized environment, these utilities often reported the total memory of the underlying physical host (the OCP node) rather than the restricted memory assigned to the specific container.

If the JVM perceives the node's total memory instead of the container's limit, it may attempt to allocate a heap size that exceeds the container's allowed threshold, resulting in an `OutOfMemoryException` or, more commonly, a `SIGKILL` by the Linux kernel's Out-Of-Memory (OOM) killer.

How the JVM Detects Container Limits

When running in a container-aware environment, the JVM inspects the control group (`cgroup`) filesystem. By reading the files exposed within the container - specifically those under `/sys/fs/cgroup/memory` - the JVM determines the hard limits imposed by the container orchestrator.

Verification Tools

To determine if your Java application is correctly detecting the container limits, you should utilize built-in JVM diagnostic flags rather than relying on standard Linux utilities like `free -m`.

> Standard utilities like `free`, `top`, or `lscpu` are **not container-aware**. They will report the resources of the host node, leading to incorrect diagnostic conclusions. Use container-aware tools, such as jcmd VM.info, and -XshowSettings:system.

Using `-XshowSettings:system`

The most direct way to verify what the JVM has detected is to pass the `-XshowSettings:system` flag during application startup or via jcmd instrumentation at runtime:

Example Output:

```
Operating System Metrics:
    Provider: cgroupv1
    Memory Limit: 128.00M
    Memory Soft Limit: unlimited
    Memory Current: 45.21M
    CPU Shares: 512
    CPU Quota: 100000
    CPU Period: 100000
```

When the JVM is container-aware, you will see explicit lines reflecting the `Memory Limit` and active CPU.

Consequently: - If the memory value matches your OpenShift/Kubernetes pod YAML limit configuration, the JVM is successfully performing container detection. - If the CPU value matches your OpenShift/Kubernetes pod YAML limit configuration at 25% so then run-java.sh is not being used and another ENTRYPOINT is being used. Otherwise, if matches 80% (or 50%) likely run-java.sh is being used.

Troubleshooting Memory Misalignment

If your application exhibits the following symptoms, the JVM is likely failing to respect container limits:

1. **Host-level reporting:** Non-container-aware commands executed inside the container (e.g., `free -m`) show memory values matching the physical node rather than the pod limit.
2. **OOM Killer termination:** The pod restarts with an exit code indicating OOM, despite the `-Xmx` setting appearing reasonable for the application's perceived workload.

Three main scenarios:

Scenario	Description & Consequences
Underutilization	The application heap is significantly smaller than the container limit because it defaulted to a generic fraction of the host's massive memory footprint, leading to frequent Garbage Collection (GC) pauses.
Overutilization	The application heap (plus off-heap) is significantly larger than the container limit, which will eventually lead: • To `OOMKilled` events • To `OutOfMemoryError` (OOMException) events • To `SIGTERM`, in case the JVM is not responsive
Misbehavior	The application runs nominally for a long time, however suddenly it misbehaves. Examples include: • **CPU Throttling:** The application was deployed without CPU boundaries and now other applications are overstepping. • **JVM stuck:** The application was deployed without CPU boundaries and now the Database is overloaded with requests. • **JVM Xmx hard-coded:** The application was deployed with `-Xmx` hard-coded; now the DevOps team cannot change the limits dynamically at runtime.

Diagnostic Strategy

To analyze resource constraints effectively:

- **Check the GC logs and pod logs:** Inspect your pod logs to confirm if `run-java.sh` or the standard OpenJDK startup scripts are being utilized, as these often contain logic to calculate heap sizes based on container limits. Same with GC logs and pod logs for misbehaviors.
- **Verify `jcmd` output:** Use `jcmd <pid> VM.info` to inspect the runtime configuration and verify `cgroup` memory controller details.
- **Compare with deployment Metadata:** Always compare the JVM's reported `Memory Limit` against the `resources.limits.memory` configured in your OpenShift `DeploymentConfig` or `Deployment` YAML and CPU with the active CPUs.

Determining CPU Availability Within Containers

In a virtualized or bare-metal host, the Java Virtual Machine (JVM) will query the operating system for the total number of physical CPUs and memory available on the host. However, in containerized environments the resources will be restricted to a subset of the host's compute resources. A container deployed by OpenShift or Kubernetes may (or may not) have container limits, a deployment without limits is called unbounded deployment.

If the JVM is not container-aware, it may incorrectly have a Xmx hard-coded or identify the host's total CPU count rather than the container's allocated limit. This mismatch can lead to inefficient thread pool sizing (such as `ForkJoinPool` or parallel garbage collection threads), causing contention and performance degradation.

How Java Detects CPU Resources

Current JDKs (Java 8u191+ and all subsequent versions) use **cgroups (Control Groups)** to determine CPU availability. The JVM reads the cgroup filesystem to calculate the `ActiveProcessorCount`.

- For cgroups v1, versions Java 8u191+ are cgroups v1 complaint.
- For cgroups v2, versions Java 8u372+ or Java 11.0.16+ are required.

Red Hat's recommendation is always to use the latest possible image and version allowed for a series of reasons.

The Calculation Logic

When running inside a container, the JVM inspects two primary metrics from the cgroup controller to determine container limits:

CPU Quota and Period (`cpu.cfs_quota_us` and `cpu.cfs_period_us`): These define the hard limit. For example, if the quota is 200,000 and the period is 100,000, the container is limited to 2 cores (200,000 / 100,000 = 2).

The JVM calculates the CPU limits of processors as the minimum of the host's processors and the calculated container limit. The JVM ignores requests (cpu.weights or cpu.shares) altogether given it does not matter for JVM’s thread calculation.

Diagnostic Tools: Container-Aware vs. Non-Aware

To verify how your Java application perceives CPU availability, you must distinguish between tools that see the Host limits versus the Container settings.

Tool Category	Examples	Insight
Non-Container Aware	`lscpu`, `nproc`	Returns the total CPU count of the **underlying OpenShift Node**. This is often misleading for capacity planning.
Container Aware	`VM.info`, `-XshowSettings:system`	Returns the `active_processor_count` as perceived by the JVM based on cgroup limits.

Verifying CPU Detection

You can inspect the JVM’s view of your container’s resources by using the `jcmd` utility.

1. Access your pod terminal: `oc rsh <pod-name>`
2. Run `jcmd` to print VM information: `jcmd <pid> VM.info`

Look for the `container` section in the output. A container-aware cgroups v1 JVM will report output similar to this:

```
container (cgroup) information:
container_type: cgroupv1
cpu_cpuset_cpus: 0-127
cpu_memory_nodes: 0-7
active_processor_count: 1  <-- JVM correctly sees 1 CPU limit
cpu_quota: -1
cpu_period: 100000
cpu_shares: 51
```

A container-aware cgroups v2 `XshowSettings` JVM will report output similar to the one below:

```
$ java -XshowSettings:system -version
Operating System Metrics:
    Provider: cgroupv2
    System not containerized.
openjdk version "21.0.12.1" 2026-08-18
OpenJDK Runtime Environment (Red_Hat-21.0.12.1.1-1) (build 21.0.12.1+1)
OpenJDK 64-Bit Server VM (Red_Hat-21.0.12.1.1-1) (build 21.0.12.1+1, mixed mode,
sharing)
```

Best Practices for CPU Configuration

- **Define Requests and Limits:** Always explicitly define `resources.limits.cpu` in your OpenShift deployment YAML. This ensures the cgroup values are populated correctly.
- **Avoid Oversizing:** If you set a high CPU limit but your application is single-threaded, the JVM may spawn excessive GC threads, leading to context-switching overhead.
- **Monitor `active_processor_count`:** If `active_processor_count` does not match your expectations, ensure you are running a supported OpenJDK container aware version.

Workaround for non-container aware JVMs in cgropus v2 hosts

As explained in how cgroups v2 impacts Java and Node.js in OpenShift 4. To understand how the transition away from cgroups v1 affects your application runtimes and also it discusses workarounds, for instance: in case you are running an older (pre-8u372 or 11.0.16-) on cgroups v2 host or using a fully non-container-aware OpenJDK the adequate workaround is two-fold:

- Manually set `-XX:ActiveProcessorCount` to match the container CPU limit. That will set the threads pools sizes.
- Manually set `-Xmx` to an appropriate value based on your container memory limit (leaving a buffer for all off-heap segments of JVM memory to avoid OOMKilled errors). That sets the heap size directly.

Cgroups does not determine (by itself) the performance of Java that is determined by the Java threads, but Java uses cgroups to set the size of the memory and the CPU limits. That's because CPU is abstracted by Java as threads, which is the main concept for running and cleaning workloads

Therefore, as long as the same QoS, same CPU settings, same context (application) is used, the performance should be the same, or very equivalent.

Specifically in terms of memory allocation, to set an appropriate Xmx, it is necessary to use memory calculations, application benchmarking, and workload testing. However, it is possible to extrapolate the memory via previous deployments and calculate from there. Example: in cgroups v1 nodes it used to consume 500mb heap inside a 1Gi container, so similar can be set i.e. Xmx for 500mb.

If there is a performance difference: other factors need to explain: Qos, Threads, number of CPUs, throttling on the cgropus v2 nodes.

Note that throttling can happen on both scenarios and it is not determined by cgroups. cgropus is just the path and the hierarchy on CPU and memory to be used.

Hands-on Exercise: Inspecting CPU Awareness in a Local Podman Environment

This hands-on exercise demonstrates how to deploy and verify Java's container CPU awareness using s2i toolkit with Podman, following the guide build OpenJDK container images locally using the standalone S2I tool.

1. Run the following commands to install s2i toolkit on the bastion VM:

   ```
   curl -LO https://github.com/openshift/source-to-
   image/releases/download/v1.3.2/source-to-image-v1.3.2-78363eee-linux-amd64.tar.gz
   tar -xzf source-to-image-v1.3.2-78363eee-linux-amd64.tar.gz
   sudo install -m 0755 s2i /usr/local/bin/s2i
   s2i version
   ```

2. Install podman and enable the associated service.

   ```
   sudo dnf install podman
   systemctl --user enable --now podman.socket
   curl --unix-socket "$XDG_RUNTIME_DIR/podman/podman.sock" http://d/_ping
   ```

   The curl command above is used to check the connectivity with the podman's socket. It should return `OK` for successful connection.

3. Use the following command to fetch a project and build an OpenJDK project locally from it via the s2i toolkit:

   ```
   git clone https://github.com/alexbarbosa1989/hotrodspringboot.git

   s2i --url "unix://$XDG_RUNTIME_DIR/podman/podman.sock" \
       --loglevel 5 \
       build hotrodspringboot/ \
       registry.access.redhat.com/ubi8/openjdk-17:latest \
       openjdk-hotrod:example
   ```

   The above source-to-image creation command may take a few minutes to complete. Successful run will show message like below:

   ```
   I0921 12:09:11.617074   25665 build.go:182] Build completed successfully
   ```

4. Confirm the image was created successfully.

   ```
   podman images
   ```

Example output:

```
[lab-user@bastion ~]$ podman images
REPOSITORY                                  TAG         IMAGE ID      CREATED
SIZE
docker.io/library/openjdk-hotrod            example     cb19badb97c0  3 minutes ago
601 MB
registry.access.redhat.com/ubi8/openjdk-17  latest      abf8175bbfa5  11 days ago
411 MB
[lab-user@bastion ~]$
```

5. Deploy application locally using podman

```
podman run -p 8081:8080 openjdk-hotrod:example
```

Success message will look like below:

```
[lab-user@bastion ~]$ podman run -p 8081:8080 openjdk-hotrod:example
Starting the Java application using /opt/jboss/container/java/run/run-java.sh ...
INFO exec -a "java" java -XX:MaxRAMPercentage=80.0 -XX:+UseParallelGC
-XX:MinHeapFreeRatio=10 -XX:MaxHeapFreeRatio=20 -XX:GCTimeRatio=4
-XX:AdaptiveSizePolicyWeight=90 -XX:+ExitOnOutOfMemoryError -cp "." -jar
/deployments/hotrodspringboot-0.0.1-SNAPSHOT.jar
INFO running in /deployments
INFO running in /deployments
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v2.7.0)
2026-09-18 14:53:36.607  INFO 1 --- [           main]
c.r.h.h.HotrodspringbootApplication      : Starting HotrodspringbootApplication
v0.0.1-SNAPSHOT using Java 17.0.20.1 on 8e34f75421fc with PID 1
(/deployments/hotrodspringboot-0.0.1-SNAPSHOT.jar started by jboss in /deployments)
. . .
. . .
2026-09-21 12:12:09.067  INFO 1 --- [           main] org.infinispan.HOTROD
: ISPN004021: Infinispan version: Infinispan 'Flying Saucer' 14.0.30.Final-redhat-
00001
2026-09-21 12:12:09.638  INFO 1 --- [           main]
o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8080 (http)
with context path '/redhat'
2026-09-21 12:12:09.652  INFO 1 --- [           main]
c.r.h.h.HotrodspringbootApplication      : Started HotrodspringbootApplication in
59.731 seconds (JVM running for 61.304)
```

The above command will not return back to the prompt. If you cannot open another terminal, run the command as background.

6. Create a new ssh session to the lab in another terminal and follow the below steps in it.
7. Check the number of CPUs available on the host machine using the `nproc` command:

   ```
   nproc
   ```

   Example output:

   ```
   [lab-user@bastion ~]$ nproc
   2
   ```

8. Check the running containers on the machine.

   ```
   podman ps
   ```

   Note the container ID from the output of the above command, as it will be used in the next steps.

   Example output:

   ```
   [lab-user@bastion ~]$ podman ps
   CONTAINER ID  IMAGE                                     COMMAND
   CREATED         STATUS         PORTS                             NAMES
   d24f38a138a2  docker.io/library/openjdk-hotrod:example  /usr/local/s2i/ru...  57
   seconds ago  Up 57 seconds  0.0.0.0:8081->8080/tcp, 8443/tcp  suspicious_archimedes
   [lab-user@bastion ~]$
   ```

9. Connect to the running container using the `podman exec` command:

   ```
   podman exec -it <container-id> /bin/bash
   ```

   Example output:

   ```
   [lab-user@bastion ~]$ podman exec -it d24f38a138a2 /bin/bash
   [jboss@d24f38a138a2 ~]$
   ```

10. List the Java processes running inside the container using the `jcmd` command:

    ```
    jcmd
    ```

Example output:

```
[jboss@d24f38a138a2 ~]$ jcmd
1 /deployments/hotrodspringboot-0.0.1-SNAPSHOT.jar
190 jdk.jcmd/sun.tools.jcmd.JCmd
```

11. Verify the CPU limits used by the application (No CPU limits defined in the container):

```
jcmd 1 VM.info | grep active_processor_count
```

Example output:

```
[jboss@d24f38a138a2 ~]$ jcmd 1 VM.info | grep active_processor_count
active_processor_count: 2
```

> The output above shows that the application is using all the CPUS available on the host machine, which an unbouded deployment and not necessarily ideal for a microservice/containerized application. To limit the CPU usage, we will now deploy the application with CPU limits.

Exit from the container's shell and return to the host machine's terminal.

```
exit
```

12. Remove the running container and deploy it again in a container with CPU limits

```
podman rm -f <container-id>

podman run -p 8081:8080 --cpus="1" openjdk-hotrod:example
```

13. Follow the above steps again to connect to the running container and verify the CPU limits used by the application.

Example output:

```
[lab-user@bastion ~]$ podman ps
CONTAINER ID  IMAGE                                     COMMAND
CREATED        STATUS        PORTS                             NAMES
2f4ba2f67633  docker.io/library/openjdk-hotrod:example  /usr/local/s2i/ru...  4
minutes ago  Up 4 minutes  0.0.0.0:8081->8080/tcp, 8443/tcp  modest_kalam
[lab-user@bastion ~]$ podman exec -it 2f4ba2f67633 /bin/bash
[jboss@2f4ba2f67633 ~]$
[jboss@2f4ba2f67633 ~]$ jcmd 1 VM.info | grep active_processor_count
active_processor_count: 1
[jboss@2f4ba2f67633 ~]$
```

The output above shows that the application is now using only 1 CPU, which is the limit we set for the container. This indicates that the application is container-aware therefore detects (and respects) the CPU limits set for its container. Java does not distinguish vCPUs from CPU, and calculates the limits from the cgroups output (quota/period).

In production applications never assign a single vCPU to a Java application in a production environment. This lab uses a single CPU only to demonstrate CPU limits for a Java process, due to resource constraints (the bastion host has only 2 CPUs). In production, allocate 2 CPUs or more to critical applications ensuring sufficient CPU resources for execution.

Hands-on Exercise: Inspecting CPU-Aware Deployments in OpenShift

This hands-on exercise demonstrates how to verify Java's container CPU awareness in OpenShift.

1. Use the following command to create a new OpenShift project:

   ```
   oc new-project hotrod-s2i-project
   ```

2. Deploy the container image with an application:

   ```
   oc new-app registry.access.redhat.com/ubi9/openjdk-17~https://github.com/alexbarbosa1989/hotrodspringboot.git --name=hotrod-app
   ```

This project and deployment are the baseline OpenShift setup for this course. Later modules reuse this same `hotrod-app` deployment and link back to this page rather than redeploying it.

In the above deployment, `oc new-app` does not set container limits, it will default to the host's limits. This directly impacts JVM heap allocation; for example, if the heap size exceeds 32 GB, Compressed Oops will automatically be disabled, and the Heap Address line will be omitted from the VM.info output.

1. Use the following command to check the status of the deployment:

   ```
   oc get deployment
   oc get pods
   ```

Wait for a few minutes until the build pod is completed and the application pod is running. You can use the `oc get pods` command to check the status of the pods.

Initially you will see the build pod running as follows.

```
[lab-user@bastion ~]$ oc get deployment
NAME         READY   UP-TO-DATE   AVAILABLE   AGE
hotrod-app   0/1     0            0           14s
[lab-user@bastion ~]$ oc get pods
NAME                 READY   STATUS    RESTARTS   AGE
hotrod-app-1-build   1/1     Running   0          19s
[lab-user@bastion ~]$
```

When the application build is finished, the final output will look like below:

```
[lab-user@bastion ~]$ oc get deployment
NAME         READY   UP-TO-DATE   AVAILABLE   AGE
hotrod-app   1/1     1            1           83s
[lab-user@bastion ~]$
[lab-user@bastion ~]$ oc get pods
NAME                          READY   STATUS      RESTARTS   AGE
hotrod-app-1-build            0/1     Completed   0          88s
hotrod-app-84fcddf456-d4kh4   1/1     Running     0          20s
[lab-user@bastion ~]$
```

2. Connect to the container running the application using the `oc rsh` command:

```
oc rsh hotrod-app-<ID>
```

Replace `<ID>` with the actual pod ID from the output of the `oc get pods` command. You will be connected to the container's shell.

Example output:

```
[lab-user@bastion ~]$ oc rsh hotrod-app-84fcddf456-d4kh4
sh-5.1$
```

3. Run the following command to see the active processor count:

```
jcmd 1 VM.info | grep active_processor_count
```

Example output:

```
sh-5.1$ jcmd 1 VM.info | grep active_processor_count
active_processor_count: 32
```

4. Run `nproc` in the same terminal to compare with OS-level tools:

```
nproc
```

Example output:

```
sh-5.3$ nproc
32
```

If you deploy a container with limits (not requests), and `active_processor_count` as reported by jcmd or java `-XshowSettings:system` matches the configured CPU limit, the application is correctly container-aware.

If it instead reports the host’s CPU values, the application is likely not container-aware. Note that nproc reports host-level CPU information and should not be used on its own to determine whether the application is container-aware.

5. Set the resource limits on the deployment.

```
oc set resources deployment/hotrod-app --limits=cpu="2"
```

6. Connect to the newly running container.

```
oc get pods
oc rsh <pod-name>
```

7. Verify the CPU limits honored by the container and the Java application running inside it.

```
nproc
jcmd 1 VM.info | grep active_processor_count
```

Example output:

```
sh-5.1$ nproc
32
sh-5.1$ jcmd 1 VM.info | grep processor_count
active_processor_count: 2
sh-5.1$
```

Notes: Given that CPU boundaries are expressed as integers in this lab, keep the following points in mind:

- Deployments created directly using `new-app` do not explicitly set CPU requests or limits. Consequently, they may run without CPU constraints unless defaults are applied by the cluster or namespace configuration.
- CPU values below 1 are effectively set to 1 CPU in this lab. This may be suitable for very small

applications, such as Quarkus workloads, but is generally not recommended for enterprise applications. Limited CPU boundaries can constrain application threads and affect garbage collection (GC) behavior. Consequently, the Serial GC may be more appropriate for workloads with limited CPU resources.

- Fractional CPU values are truncated, not rounded, when converted to integer CPU boundaries. For example, both 2200 millicores and 2950 millicores result in 2 CPUs.
- Java does not use CPU requests to determine its available processor count. It relies on CPU limits when container awareness is enabled and the runtime correctly detects the container's resource constraints.
- Java workloads can benefit from explicitly configured CPU and memory limits for more predictable resource usage and performance. See `unbounded deployment` for further details.
- In the VM.info file, the line `Heap address: 0x00000000e0000000, size: MB`, Compressed Oops mode: 32-bit, will only appear if the heap is lower than 30Gb, otherwise `Compressed Oops` will be disabled and the line won't be there. Set `-XX:+UseCompressedOops` manually in case the heap is higher than 30Gb to see that line.

Risks and Memory Management

When deploying Java in containerized environments, improper memory configuration can lead to significant stability and performance issues. Understanding the balance between heap and off-heap memory is critical to preventing infrastructure-level failures.

- **Decoupling Risks:** Hard-coding heap sizes (e.g., via `-Xmx`) without considering container limits can lead to memory mismatches. If the JVM ignores container boundaries, it may trigger cgroups OOM-Kills, causing the kernel to terminate the application.
- **Off-Heap Considerations:** Container-aware Java calculates heap size as a percentage of available container memory. It is essential to account for off-heap usage (e.g., metaspace, thread stacks, native buffers) to ensure the total memory footprint does not exceed the container limit.
- **Resource Underutilization:** Inadequate configuration can lead to inefficient resource usage, where the JVM reserves significantly less memory than the container provides, leaving allocated infrastructure resources idle.
- **Non-container aware allocation:** Allocating a certain percentage for JVM container, but not benchmarking/tracking non-container aware components inside the Java application. For example, jemalloand glibc's malloc that are below the JVM and are both not container aware.

Best Practices:

- Prioritize container-aware JVM settings over manual heap overrides to allow for dynamic scaling.
- Benchmark application memory usage to determine the optimal ratio between heap and off-heap requirements.
- Avoid configurations that ignore cgroup limits, as these bypass the self-tuning capabilities of modern JDKs.
- Track specific percentage utilization for the application.
- Use limits in the container settings to make sure Java won't use host details.
- Understand Quality of Service (QoS) and how it impacts (Java) deployments.

Red Hat OpenJDK's ENTRYPOINT Details:

Default OpenJDK container uses run-java.sh as ENTRYPOINT, this has several implications:

- MAXRAMPercentage is set at 80%
- ParallelGC is the default container (regardless of CPU)
- Environment variables: `JAVA_OPTS`, `JAVA_OPTS_APPEND`
- PID 1 will be the default Java process

Example:

```
$ sudo docker run -p 8081:8080 --cpus="2" openjdk-hotrod:example
[sudo] password for fdemeloj:
```

```
Starting the Java application using /opt/jboss/container/java/run/run-java.sh ...
INFO exec -a "java" java -XX:MaxRAMPercentage=80.0 -XX:+UseParallelGC
-XX:MinHeapFreeRatio=10 -XX:MaxHeapFreeRatio=20 -XX:GCTimeRatio=4
-XX:AdaptiveSizePolicyWeight=90 -XX:+ExitOnOutOfMemoryError -cp "." -jar
/deployments/hotrodspringboot.jar
```

Implications of Decoupling Heap Memory from Container Limits

Inside a containerized environment, the JVM's ability to detect container resource boundaries is a critical feature. When developers manually specify heap sizes using the traditional heap limit: `-Xmx` without regard for the container's cgroup limits, they risk decoupling the JVM memory footprint from the orchestration layer. This decoupling often leads to significant operational instability and an inherited risk of OOMKill.

The Risks of Manual Overrides

Before Java became fully container-aware, it was common practice to explicitly define `-Xmx` (maximum heap size) to ensure the JVM did not attempt to consume all available memory on a physical host. For example, on a straight VM deployment. However, in a containerized world, this approach creates several hazards:

- **cgroup OOMKills:** If the `-Xmx` is set too high relative to the container's memory limit, the total memory usage—heap plus off-heap (metaspace, thread stacks, GC structures, and native buffers)—will eventually exceed the cgroup limit. The container runtime or the kernel will detect this violation and trigger an `OutOfMemoryKill` (OOMKill), terminating the process abruptly.
- **Resource Underutilization:** If the `-Xmx` is set significantly lower than the container limit, the application cannot scale its memory usage to handle increased load, even if the container has plenty of headroom. This results in inefficient resource allocation and potential `OutOfMemoryException` (OOME) errors within the JVM, despite the container appearing to have available memory.
- **Off-heap limits:** DevOps teams/developers may often focus exclusively on the heap. However, native memory (off-heap) usage is not governed by `Xmx`, although `Xmx` caps the heap limit. The JVM is a combination of both heap and off-heap, and both segments together potentially cause the combined memory pressure to breach the container's hard limit.

Build the container with Xmx (hard-coded manual setting) is an Anti-Pattern

Building an application with JVM settings hard coded is an anti-pattnern. That's because a change on the JVM flag would require a new build, which breaks the main idea of Kubernetes.

In other words, baking JVM configurations directly into an application artificat is an anti-pattern that severely limits the flexibility of the application directly and impacts its deployment.

For Red Hat's OpenJDK containers, the run-java.sh allows `JAVA_OPTS` and `JAVA_OPTS_APPEND` environment variables that allow the user to set JVM flags, including MaxRAM Percentage and Garbage collector.

Decoupling is an Anti-Pattern

Decoupling occurs when the JVM's memory configuration ignores the orchestrator's resource constraints. This makes the application blind to the environment it resides in.

Current Java versions (Java 8u191+, Java 11.0.16+, Java 17+) are designed to be fully container-aware on both cgroups v1 and cgroups natively, meaning they automatically derive memory settings from cgroups. Overriding this by hardcoding heap values removes the benefits of this feature, essentially reverting the JVM to a legacy mode that is non-container aware of its surroundings.

Best Practices: The Percentage-Based Approach

To avoid decoupling, the recommendation is to shift from absolute values (`-Xmx`) to relative values. The percentage already comes by default and the user can tune that via `-XX:MaxRAMPercentage`, the JVM maintains a coupled relationship with the container:

- **Dynamic Changes:** As you increase the memory limit of your OpenShift or Kubernetes pod, the JVM heap will not automatically adjust to maintain the defined percentage. That's because the JVM only detects the limits once when it starts and therefore needs to fully restart to detect its limits, so pod restart/JVM start are required.
- **Safety Margin:** Leaving a percentage of the container memory free (i.e., not allocating 100% to the heap) provides the necessary space for off-heap components—such as Netty buffers, thread stacks, and Metaspace—preventing cgroup-level OOMKills.

If you must explicitly define memory, use `-XX:MaxRAMPercentage=N`. This ensures the heap size is always a derivative of the container limit, preserving the coupling between the JVM and the infrastructure.

Summary of Implications

Risk Factor	Consequence	Impact
Fixed -Xmx	Decoupling from cgroups	Unexpected OOMKills
Ignored Off-Heap	Native memory starvation	JVM instability / Crashes
Manual Constraints	Operational rigidity	Need for manual tuning during scaling

By allowing the JVM to detect container limits automatically, i.e. by utilizing percentage-based configurations, the DevOps teams ensure the Java application remains dynamic, resilient, portable, and correctly scaled within your cluster limits and boundaries.

Managing Heap and Off-Heap Memory in Containerized Environments

In current Java applications (Java 8+), memory management extends beyond the Java Heap. While the Heap is responsible for storing object instances, the **Off-Heap** memory—encompassing Metaspace, Code Cache, Thread stacks, and Direct ByteBuffers. Those are critical for the stability of containerized deployments.

When a Java application runs in a container, it is subject to strict Linux Control Group (cgroup) memory limits. If the sum of the Heap and Off-Heap memory exceeds the container limit, the container runtime will trigger an `OOMKill` (Out of Memory Kill).

Java ENTRYPOINT Practices:

Default OpenJDK container uses run-java.sh as ENTRYPOINT, this has several implications.

Feature	`run-java.sh` Entry Point	Straight `java` Start
Execution Method	Shell wrapper script that evaluates the environment and builds the Java command line.	Native binary execution directly invoking the JVM.
Environment Variables	Automatically parses custom variables like `JAVA_OPTS_APPEND`, `JAVA_OPTS`	Ignores script-specific variables. Relies strictly on command-line arguments and `JAVA_TOOL_OPTIONS` and `_JAVA_OPTIONS`.
Default JVM Tuning	Automatically applies container-friendly defaults (e.g., `-XX:MaxRAMPercentage`, ParallelGC settings) if not overridden.	Uses standard JVM defaults unless explicitly defined in the command line or deployment manifest.
GC collector	Run-java.sh sets ParallelGC as default collector	The collector is detected based on CPU and the JDK version (1 CPU will set SerialGC, 2+ on JDK 11+ sets G1GC).
Container Awareness	Container aware: sets MaxRAMPercentage at 80% (or 50% previously).	Container aware: sets MaxRAMPercentage at 25%, which is the default value upstream
Process Management	It can have Java process with different PID than 1, but not as a rule	Directly executes as PID 1 given entrypoint will be the first process to be executed
Recommendation	run-java.sh is to be used: it gives flexibility.	Straight deployment should be avoided, reason: you cannot tune MAxRAMPercentage as easily and may be bounded by the 25% heap limit, which is the full idea of the env variables.

In a table with the direct implications for deployment:

<table>
<tr><th><code>run-java.sh</code> ENTRYPOINT (Red Hat Default)</th><th><code>java</code> ENTRYPOINT (Direct)</th></tr>
<tr><td>

Built-in wrapper script provided by Red Hat UBI Java images.
Automatically detects runnable JAR files in the <code>/deployments</code> directory.
Parses specific environment variables like <code>JAVA_OPTS_APPEND</code>.
Automatically calculates JVM memory limits based on container size if not explicitly set.

</td><td>

Bypasses the Red Hat wrapper script, executing the Java binary directly.
Requires manually defining the working directory and execution command.
Requires using standard Java variables (like <code>JAVA_TOOL_OPTIONS</code>) or passing arguments directly in the ENTRYPOINT to tune the JVM.
Does not provide automatic memory calculation based on container bounds.

</td></tr>
<tr><td>
<pre>
FROM registry.access.redhat.com/ubi9/openjdk-17:latest

Copy the compiled JAR into the standard deployments folder
COPY target/hotrod-app.jar /deployments/hotrod-app.jar

No ENTRYPOINT required; the base image automatically executes run-java.sh
</pre>
</td><td>
<pre>
FROM registry.access.redhat.com/ubi9/openjdk-17:latest

Create a working directory and copy the JAR
WORKDIR /app
COPY target/hotrod-app.jar /app/hotrod-app.jar

Explicitly define the Java entrypoint
ENTRYPOINT ["java", "-jar", "hotrod-app.jar"]
</pre>
</td></tr>
</table>

References for environment variables

For comprehensive details on configuring Red Hat Universal Base Image (UBI) OpenJDK containers, including supported environment variables across different UBI versions, consult the official documentation:

- Red Hat UBI OpenJDK Container Images Documentation

Cgroups v1 vs cgropus v2

By default Java OpenJDK detects the container limits via cgroups path, which change according to the version, as below. However, there are specific use cases such as JBoss EAP related products, which initialization scripts may overwrite OpenJDK detection as explained on What Red Hat middleware software is cgroups v2 compatible?.

Table 2. cgroups v1 vs. v2 Memory Metrics

Metric	cgroups v1 Path	cgroups v2 Path
Peak Usage	`/sys/fs/cgroup/memory/memory.max_usage_in_bytes`	`/sys/fs/cgroup/memory.peak` (Requires Kernel 5.19+)

Metric	cgroups v1 Path	cgroups v2 Path
Current Usage	`/sys/fs/cgroup/memory/memory.usage_in_bytes`	`/sys/fs/cgroup/memory.current`
Hard Limit	`/sys/fs/cgroup/memory/memory.limit_in_bytes`	`/sys/fs/cgroup/memory.max`
Soft Limit	`/sys/fs/cgroup/memory/memory.soft_limit_in_bytes`	`/sys/fs/cgroup/memory.high`

The Decoupling Risk

Historically, developers focused exclusively on `-Xmx` (maximum heap size). However, in a container, the JVM sees the container's memory limit. If you configure the Heap to consume too large a percentage of the container's limit, you leave insufficient headroom for Off-Heap requirements. Therefore a percentage approach is used:

Key Off-Heap components that must be accounted for include:

- **Metaspace:** Stores class metadata.
- **Code Cache:** Stores compiled native code by the JIT compiler.
- **Thread Stacks:** Each thread requires native memory; in applications with high thread counts, this can lead to significant memory consumption.
- **Direct Buffers:** Used by NIO libraries and netty-based frameworks.

Best Practices

To prevent `OOMKills`, you must treat the container's memory limit as the total capacity for both Heap and Off-Heap.

1. **Calculate the Total Footprint:** Before setting heap percentages, perform load testing to determine the baseline Off-Heap usage of your application under peak load.
2. **Use Relative Sizing:** Instead of fixed `-Xmx` values, JVM will already start with a percentage value that leverage container-aware JVM flags that allow the heap to scale proportionally to the container limit. Therefore, user can tune that value:

   ```
   -XX:MaxRAMPercentage=80.0 (default value)
   ```

   Note: If your application is highly dependent on Off-Heap (e.g., uses large Direct Buffers or high thread counts), you should reduce this percentage to 50% or 60% to ensure enough spare space/breathing room for native memory allocations.

3. **Monitor Native Memory:** Use the Native Memory Tracking (NMT) feature to gain visibility into non-heap memory usage.

   ```
   # Enable NMT in the container
   -XX:NativeMemoryTracking=summary (or detail)
   ```

Hands-on Activity: Analyzing Heap and Off-Heap Settings

This activity guides you through identifying the memory footprint of your Java process.

1. Use `jcmd` to inspect the memory distribution of the running containerized application at runtime.
2. Assuming the same application as before — the one deployed in Inspecting CPU-Aware Deployments in OpenShift.

   ```
   oc get deployment
   oc get pods
   ```

   Example outpput:

   ```
   [lab-user@bastion ~]$ oc get deployment
   NAME         READY   UP-TO-DATE   AVAILABLE   AGE
   hotrod-app   1/1     1            1           2m16s
   [lab-user@bastion ~]$
   [lab-user@bastion ~]$ oc get pods
   NAME                          READY   STATUS      RESTARTS   AGE
   hotrod-app-1-build            0/1     Completed   0          12m
   hotrod-app-7569b6c6cd-52dqg   1/1     Running     0          4m28s
   [lab-user@bastion ~]$
   ```

3. Connect to the running container using the `oc rsh` command:

   ```
   oc rsh <pod-name>
   ```

4. Confirm the default MaxRAMPercentage value is at 80% via jcmd VM.info's `Command Line`:

   ```
   jcmd 1 VM.info | grep 'Command Line'
   ```

   Example output:

   ```
   sh-5.1$ jcmd 1 VM.info | grep 'Command Line'
   Command Line: -XX:MaxRAMPercentage=80.0 -XX:+UseParallelGC -XX:MinHeapFreeRatio=10
   -XX:MaxHeapFreeRatio=20 -XX:GCTimeRatio=4 -XX:AdaptiveSizePolicyWeight=90
   -XX:+ExitOnOutOfMemoryError /deployments/hotrodspringboot-0.0.1-SNAPSHOT.jar
   sh-5.1$
   ```

   Note `-XX:MaxRAMPercentage=80.0` set by default in the run-java.sh script. This means that the JVM will use up to 80% of the container's memory limit for the Java Heap.

5. Check the Heap Max Capacity, container type, and memory limit in bytes:

```
jcmd 1 VM.info | grep -e 'Heap Max Capacity' -e 'container_type' -e
'memory_limit_in_bytes'
exit
```

Example output:

```
sh-5.1$ jcmd 1 VM.info | grep -e 'Heap Max Capacity' -e 'container_type' -e
'memory_limit_in_bytes'
 Heap Max Capacity: 103040M <------------------- 103G (80% of 128G)
container_type: cgroupv2 <----------------------------------------- verifying
cgroups v2
memory_limit_in_bytes: unlimited <------------------ no memory limit set
sh-5.1$ exit
[lab-user@bastion ~]$
```

6. Check the memory capacity of the node running the pod using the `free` command:

```
oc get nodes -o wide
oc debug node/<node_name> -- bash -c "free -m"
```

Replace <node_name> with the name of the node where your pod is running.

Example output:

```
[lab-user@bastion ~]$ oc get nodes
NAME                            STATUS   ROLES                         AGE
VERSION
control-plane-cluster-phzlt-1   Ready    control-plane,master,worker   22h
v1.34.9
[lab-user@bastion ~]$
[lab-user@bastion ~]$ oc debug node/control-plane-cluster-phzlt-1 -- bash -c "free
-m"
Temporary namespace openshift-debug-vq288 is created for debugging node...
Starting pod/control-plane-cluster-phzlt-1-debug-72n7v ...
To use host binaries, run `chroot /host`. Instead, if you need to access host
namespaces, run `nsenter -a -t 1`.
               total        used        free      shared  buff/cache   available
Mem:          128797       21721       55812         273       52770      107075
Swap:              0           0           0

Removing debug pod ...
Temporary namespace openshift-debug-vq288 was removed.
[lab-user@bastion ~]$
```

Note the total memory of the node is 128G, which matches the Heap Max Capacity of 103G (80% of 128G) seen in the previous step.

ℹ Although a container may have CPU limits configured, without an explicit memory limit, it can use memory without a defined container-level constraint. In this case, Java determines its memory limit based on the host's available memory, with MaxRAMPercentage set to 80% of the host memory.

7. Let's set a limit on the container memory and verify the percentage.
8. Set 2Gi limit on the container and leave the MaxRAMPercentage at default 80%

```
oc set resources deployment/hotrod-app --limits=memory="2Gi"
```

9. Get the pod name and go inside the pod:

```
oc get pod
oc rsh <pod-name>
```

10. Check the Heap Max Capacity and memory limit in bytes:

```
jcmd 1 VM.info | grep -e 'Heap Max Capacity' -e 'memory_limit_in_bytes'
exit
```

Example output:

```
sh-5.1$ jcmd 1 VM.info | grep -e 'Heap Max Capacity' -e 'memory_limit_in_bytes'
 Heap Max Capacity: 1640M <------------------- 1640M (80% of 2Gi)
memory_limit_in_bytes: 2097152 k <------------------ 2Gi limit set on the container
sh-5.1$
```

11. Assuming 80% is an inadequate value, let's set the MaxRAMPercentage to 50% and verify the Heap Max Capacity again.

```
oc set env deployment/hotrod-app JAVA_OPTS_APPEND="-XX:MaxRAMPercentage=50.0"
oc get pod
oc rsh <pod-name>
```

12. Check the MaxRAMPercentage, Heap Max Capacity and memory limit in bytes:

```
jcmd 1 VM.info | grep -e 'Command Line' -e 'Heap Max Capacity' -e
'memory_limit_in_bytes'
exit
```

Example output:

```
sh-5.1$ jcmd 1 VM.info | grep -e 'Command Line' -e 'Heap Max Capacity' -e
```

```
'memory_limit_in_bytes'
Command Line: -XX:MaxRAMPercentage=80.0 -XX:+UseParallelGC -XX:MinHeapFreeRatio=10
-XX:MaxHeapFreeRatio=20 -XX:GCTimeRatio=4 -XX:AdaptiveSizePolicyWeight=90
-XX:+ExitOnOutOfMemoryError -XX:MaxRAMPercentage=50.0
/deployments/hotrodspringboot-0.0.1-SNAPSHOT.jar
 Heap Max Capacity: 1G
memory_limit_in_bytes: 2097152 k
sh-5.1$ exit
[lab-user@bastion ~]$
```

Note that the `Command Line` shows both `-XX:MaxRAMPercentage=80.0` and `-XX:MaxRAMPercentage=50.0`. The latter takes precedence, resulting in a Heap Max Capacity of 1G (50% of 2Gi).

13. For more details on off-heap settings, deploy with NativeMemoryTracking (NMT) as below:
14. Deploy the application with NativeMemoryTracking summary (or details)

```
oc set env deployment/hotrod-app JAVA_OPTS_APPEND="-
XX:NativeMemoryTracking=summary"
```

15. Connect to the running container using the `oc rsh` command

```
oc get pod
oc rsh <pod-name>
```

16. Confirm the application has the correct JVM argument, NativeMemoryTracking, applied:

```
jcmd 1 VM.info | grep jvm_args
```

Example output:

```
sh-5.1$ jcmd 1 VM.info | grep jvm_args
jvm_args: -XX:MaxRAMPercentage=80.0 -XX:+UseParallelGC -XX:MinHeapFreeRatio=10
-XX:MaxHeapFreeRatio=20 -XX:GCTimeRatio=4 -XX:AdaptiveSizePolicyWeight=90
-XX:+ExitOnOutOfMemoryError -XX:NativeMemoryTracking=summary <-------------- HERE
sh-5.1$
```

17. Full example output for VM.native_memory summary:

```
sh-5.3$ jcmd 1 VM.native_memory summary
1:
$ jcmd 1 VM.native_memory
1:
Native Memory Tracking: (Omitting categories weighting less than 1KB)
Total: reserved=28725786KB, committed=1518126KB
```

```
       malloc: 62206KB #254051
       mmap:   reserved=28663580KB, committed=1455920KB

-                 Java Heap (reserved=26286080KB, committed=514048KB)
                            (mmap: reserved=26286080KB, committed=514048KB)

-                     Class (reserved=1049363KB, committed=5779KB)
                            (classes #8643)
                            (  instance classes #8081, array classes #562)
                            (malloc=787KB #18256) (at peak)
                            (mmap: reserved=1048576KB, committed=4992KB)
                            (  Metadata:   )
                            (    reserved=65536KB, committed=32256KB)
                            (    used=32053KB)
                            (    waste=203KB =0.63%)
                            (  Class space:)
                            (    reserved=1048576KB, committed=4992KB)
                            (    used=4780KB)
                            (    waste=212KB =4.25%)

-                    Thread (reserved=39016KB, committed=2836KB)
                            (thread #39)
                            (stack: reserved=38912KB, committed=2732KB)
                            (malloc=61KB #232) (peak=72KB #242)
                            (arena=43KB #74) (peak=471KB #42)

-                      Code (reserved=248583KB, committed=14187KB)
                            (malloc=895KB #5162) (peak=895KB #5163)
                            (mmap: reserved=247688KB, committed=13292KB)

-                        GC (reserved=989213KB, committed=905329KB)
                            (malloc=28853KB #134) (peak=28861KB #136)
                            (mmap: reserved=960360KB, committed=876476KB)

-                  Compiler (reserved=195KB, committed=195KB)
                            (malloc=30KB #355) (peak=66KB #139)
                            (arena=165KB #5) (peak=54165KB #23)

-                  Internal (reserved=12650KB, committed=12650KB)
                            (malloc=12614KB #5439) (peak=12741KB #5348)
                            (mmap: reserved=36KB, committed=36KB)

-                     Other (reserved=4892KB, committed=4892KB)
                            (malloc=4892KB #20) (peak=4902KB #22)

-                    Symbol (reserved=9203KB, committed=9203KB)
                            (malloc=8524KB #220921) (at peak)
                            (arena=680KB #1) (at peak)
```

If you see "Native memory tracking is not enabled", you must enable it by adding the following

JVM option:

- Option 1 - `-XX:NativeMemoryTracking=summary`
- Option 2 - `-XX:NativeMemoryTracking=details`

18. Update the current deployment with this setting

```
oc set env deployment/hotrod-app JAVA_OPTS_APPEND="-XX:NativeMemoryTracking=detail"
```

19. Access the terminal of your new running pod:

```
oc rsh <pod-name>
```

20. Inspect the memory distribution:

```
jcmd 1 VM.native_memory summary
```

Example output:

```
sh-5.3$ jcmd 1 VM.native_memory summary
Picked up JAVA_OPTS_APPEND: -XX:NativeMemoryTracking=summary
1:
Native Memory Tracking:
(Omitting categories weighting less than 1KB)

Total: reserved=33545316KB, committed=288676KB
       malloc: 59876KB #80575, peak=111544KB #81451
       mmap:   reserved=33485440KB, committed=228800KB

-                 Java Heap (reserved=31424512KB, committed=147456KB)
                            (mmap: reserved=31424512KB, committed=147456KB, at
peak)

. . .

-      String Deduplication (reserved=1KB, committed=1KB)
                            (malloc=1KB tag=String Deduplication #8) (at peak)

-           Object Monitors (reserved=75KB, committed=75KB)
                            (malloc=75KB tag=Object Monitors #385) (at peak)
```

Additional Notes:

- To analyze the output for off-heap, look at sections such as:
  - **Internal:** Memory used by the JVM internal structures.

  - **Symbol:** Memory used for string interning.
  - **Thread:** Memory consumed by thread/stack spaces.
- To ensure your configuration is resilient, verify that your application does not exceed the container limit.
- View the current cgroup memory limit for your container:

```
### CGROUPS V1
cat /sys/fs/cgroup/memory/memory.limit_in_bytes
### CGROUPS V2
cat /sys/fs/cgroup/memory.max
```

- Compare this value against your calculated Heap + Off-Heap usage. If your application's `Total` committed memory (as seen in NMT) + heap usage exceeds this limit, one must adjust your `MaxRAMPercentage` downward or increase the container's memory limit in OpenShift `Deployment` YAML.

Troubleshooting Tip:

The following risks are possible on the deployment in orchestrators like Openshift 4:

- **OOMKill**: The full memory is above the container size or cgroups v2 detection is failing.
- **OOMException**: Usually off-heap allocation or metaspace, not that if `ExitOnOutOfMemoryError` flag is enabled, so then the JVM will exit.
- **SIGTERM**: In case communication is impacted or the JVM is unresponsive sigterm can kick in. That will be in the events.

If you encounter `cgroup OOMKills` despite having a small Java Heap, use `jcmd` to identify the category consuming unexpected amounts of native memory. From that, investigate off-heap components such as non-container aware libraries, thread pool sizing, and do an investigation from the off-heap size perspective.

Best Practices for Java Memory Configuration in Containers

As Java has evolved to be flexible and container-aware, some practices could be considered anti-patterns, such as:

- `Xmx` direct usage: (in most cases) is an anti-pattern. By default Java will have already percentage utilization inside a container. In other words, the manual strategy of defining absolute heap sizes (e.g., `-Xmx2g`) has become a deployment anti-pattern, because it decouples from the container size.
- JVM arguments directly at build time, rather than runtime. For example, avoid the usage of JVM arguments hardcoded in a Containerfile (Dockerfile), rather than customizable in a deployment resource (YAML), which is more adequate.

When the JVM is unaware of its container boundaries, it will default to host-level metrics, leading to resource contention, poor performance, or premature `cgroups` OOM-Kills.

The current approach focuses on **relative size configuration** so the size of Java's heap changes based on the container size (dynamically requiring restarting). This maintains a healthy balance between the JVM heap and the native (off-heap) memory required by the container environment.

Moving from Absolute to Relative Configuration

To ensure your Java application scales gracefully within an OpenShift or Kubernetes pod, follow these best practices for memory management.

1. Use Percentage-Based Flags

Container awareness comes by default, therefore: instead of hardcoding memory values, utilize percentage-based flags. These ensure the JVM dynamically adjusts its heap size relative to the memory limits defined in your Pod specification.

- `-XX:MaxRAMPercentage=N`: Set by default. Defines the maximum heap size as a percentage of the container's memory limit. For most applications, a value between `50.0` and `80.0` is usually adequate, leaving the remaining space for off-heap requirements. Off-heap heavy application may require a low percentage.
- `-XX:InitialRAMPercentage=N`: Sets the initial heap size as a percentage of the container limit.
- `-XX:MinRAMPercentage=N`: Sets the minimum heap size when the container limit is very small.

2. Understanding the Heap and off-Heap allocation

The memory not occupied by the JVM heap is not wasted — it is necessary for the stability of the Java application. When you configure your heap, you must leave sufficient headroom for:

- **Dynamic libraries and instrumentation**: Dynamic libraries and observability tools will take off-heap space
- **Metaspace**: Stores class metadata.
- **Code Cache**: Stores JIT-compiled native code.
- **Thread Stacks**: Each thread requires native memory; a large number of threads can lead to unexpected native memory pressure.
- **Direct Byte Buffers**: Often used by frameworks like Netty, AMQ, or Data Grid.
- **Garbage Collector Metadata**: The structures required by G1GC or ZGC to track object references.

Best Practice: If your application relies heavily on off-heap memory (e.g., streaming or high-throughput messaging), use a lower `MaxRAMPercentage` (e.g., 50%) to prevent the total container usage from hitting the `cgroups` limit.

3. Avoid MaxRAM Override

Avoid setting the `-XX:MaxRAM` flag. When you set `MaxRAM` one effectively decouple the JVM's view of memory from the actual container `cgroups` limit. This removes completely the purpose of container awareness and can lead to the JVM attempting to allocate more memory than the container is permitted to use, resulting in an immediate OOM-Kill by the kernel. `MaxRAM` sets the full size of the heap and off-heap and it is very useful in specific cases.

4. Aligning with Container Orchestration

Ensure your Pod resource requests and limits are configured to support your Java workload:

- **Memory Limits**: Always set a memory limit in your Deployment/DeploymentConfig. The JVM uses this value to calculate the heap size when using percentage-based flags.
- **Avoid Oversizing**: Do not set the limit significantly higher than the application's actual footprint. While Kubernetes handles overcommitment, the JVM will calculate its heap based on the **limit**, not the actual usage.

5. Understand Red Hat OpenJDK's run-java.sh specifics.

- **environment variables**: run-java.sh sets default environment varaiables that the user can use at runtime (in the container env variables) to control the Java deployment at runtime
- **GC collector**: run-java.sh sets default ParallelGC
- **JVM settings**: run-java.sh sets MaxRAMPercentage at 80% (currently) rather than relying on 25% default value.

Summary Table: Configuration Comparison

Approach	Recommendation
`-Xmx` (Absolute)	**Discouraged**. Will cause a decoupling and potential OOM-Kills. Also any change will cause problems. Xmx overwrites any percentage (MaxRAMPercentage). Xmx sets full heap size.
`-XX:MaxRAMPercentage`	**Default/Recommended**. Automatically aligns heap with container `cgroups` limits.
`MaxRAM`	**Avoid**. Similar to hard coding Xmx, it decouples memory views between the OS and the JVM. MaxRAM sets full heap+off-heap size.

When using Red Hat OpenJDK images, the provided `run-java.sh` initialization script sets a specific `MaxRAMPercentage` percentage, currently at 80%. Previously, the MaxRAMPercentage was 50%. This value sets the Max Heap at 80% of the container size. The remainder goes to off-heap. Overriding the `MaxRAMPercentage` with a custom value is sufficient to customize the deployment, considering that 80% heap may not be adequate. If the scripts run-java.sh is not used, so the default percentage (MaxRAMPercentage) will be 25% (for heap) so 75% for off-heap.

Troubleshooting Resource Constraints

If your application is experiencing `OutOfMemoryException` or being killed by the container orchestrator, analyze the environment with the following diagnostic flow:

1. **Check `VM.info` file**: Use Java Diagnostics (jcmd) to print all details of Java concerning containers.
2. **Orchestrator and host details**: Verify the events, deployment yaml status, and host logs (sos_report), which will record OOMKill events.
3. **Check `cgroups` usage**: Use `cat /sys/fs/cgroup/memory/memory.stat` inside the container to see if the container is hitting its hard limit. This file is usually used for Prometheus/cadAdvisor output scrapping.
4. **Analyze GC logs**: Verify `GC logs` about the cgroups details, GC allocation, and GC collector behavior.
5. **Analyze Native Memory Tracking (NMT)**: Enable `-XX:NativeMemoryTracking=summary` (or `details`) to help identify memory leaks or native memory segments allocation.
6. **Benchmark**: Always performance-test with a memory profiler before finalizing your percentage values to ensure the off-heap headroom is sufficient for peak load.

Garbage collectors

Although the core of this course is not to discuss Java's garbage collector, it does offer a possibility of discussing different GC collectors. As mentioned when using run-java.sh ENTRYPOINT (default ENTRYPOINT for Red Hat OpenJDK images), the default collector is ParallelGC however others can be used given enough CPUs. The discussion will depend on the CPU availability and application profile.

Garbage Collector	JVM Flag	Description & Best Fit
Serial GC	`-XX:+UseSerialGC`	Uses a single thread for garbage collection. **Best for single-core containers** or microservices with very small heaps (< 100MB) where minimal GC overhead and memory footprint are required.
Parallel GC	`-XX:+UseParallelGC`	Focuses on maximizing throughput by using multiple threads to perform GC, but stops the application completely during collection. **Best for batch processing** or applications where latency (pause times) is not a critical concern.
G1 GC (Garbage-First)	`-XX:+UseG1GC`	The default GC since Java 9. Balances throughput and latency by dividing the heap into regions. **Best for general-purpose web applications** running on multi-core containers with heaps larger than 2GB.
ZGC (Z Garbage Collector)	`-XX:+UseZGC`	An ultra-low latency GC designed to keep pause times under 1 millisecond. **Best for latency-sensitive applications**, though it may require slightly more CPU and memory overhead. (Production ready since Java 15).

Garbage Collector	JVM Flag	Description & Best Fit
Shenandoah GC	`-XX:+UseShenandoahGC`	Red Hat’s ultra-low latency GC that performs evacuation work concurrently with the running Java threads. **Best for highly responsive applications** requiring consistent, predictable pause times regardless of heap size. For more details, see the Beginner’s guide to Shenandoah Garbage Collector.

OpenShift and Kubernetes Integration

In containerized environments, Java applications rely on integration with OpenShift and Kubernetes to manage resource allocation through cgroups. Understanding how these platforms govern resources is critical for stable deployments:

- **Resource Specifications:** Kubernetes uses `requests` and `limits` defined in deployment YAML to manage compute resources. `Requests` guide the scheduler for node placement, while `limits` enforce hard constraints on CPU and memory usage. Java only cares about limits.
- **Cgroup Enforcement:** Platform resource definitions translate directly into Linux kernel cgroup values. CPU limits define execution quotas, while memory limits act as the boundary for the container's total memory footprint.
- **QoS Classes:** OpenShift employs Quality of Service (QoS) classes to handle resource contention. There are three classes: Guaranteed, Burstable, and Best-effort (in order of eviction). While Guaranteed QoS provides stable resource allocation always, it does not prevent cgroup-level OutOfMemory (OOM) kills or OOMException from happening, just puts the application at the end of the eviction process (after burstable and best-effort).
- **Operational Alignment:** Proper integration requires aligning Java heap configurations with container-level constraints to prevent discrepancies between the JVM's view of memory and the actual limits imposed by the OpenShift platform.

Java only cares about limits, so if the deployment does not have limits, it will use the host as limits. Unbouded deployment is a type of deployment that will use the full Host limits.

Configuring Limits and Requests in Red Hat OpenShift

In the context of Red Hat OpenShift Container Platform (OCP), effectively managing resources is a balancing act between scheduling efficiency and runtime performance. To optimize Java applications, it is critical to distinguish how OCP handles resource definitions and how those definitions translate to the Linux kernel and the Java Virtual Machine (JVM).

The Relationship Between Requests and Limits

OpenShift utilizes Kubernetes-native resource management, defined within the container specification of a Deployment or Pod YAML.

- **Requests:** These are used by the OpenShift scheduler to place a Pod on a node. They represent the minimum amount of CPU and memory guaranteed to the container. If a node does not have enough unallocated resources to satisfy the request, the scheduler will not place the Pod on that node.
- **Limits:** These define the hard ceiling for resource consumption. If a container exceeds its memory limit, it risks being terminated by the kernel (OOMKilled). CPU limits act as a throttling mechanism, restricting the container's execution time within a defined period.

The above will impact also the Quality of Service, QoS, of the pods. The QoS is used directly for eviction threshold:

QoS Class	Configuration Criteria	Eviction Priority
Guaranteed	Every container in the Pod must have both memory and CPU `limits` and `requests` configured, and the `requests` must equal the `limits`.	**Lowest Priority** (Last to be evicted during node resource pressure).
Burstable	The Pod does not meet the criteria for Guaranteed, but at least one container has a memory or CPU `request` defined. (e.g., `requests` are lower than `limits`, or limits are not set).	**Medium Priority** (Evicted if no BestEffort pods remain and the node is under pressure).
BestEffort	No containers in the Pod have any memory or CPU `requests` or `limits` defined.	**Highest Priority** (First to be evicted when the node runs out of resources).

Resource Allocation Table

When running Java inside a container, it is critical to understand how the container engine translates CPU configurations into Linux cgroup metrics. The following table summarizes the differences between requests and limits, highlighting a key behavior: the JVM calculates its available processors based solely on hard CPU limits, completely ignoring CPU requests. This value is then used for the thread calculation.

Resource Type	Purpose	Calculation Method	JVM Behavior
CPU Request	Scheduling and node assignment.	Sets `cpu.share` in cgroups to prioritize usage.	Java does not use requests.
CPU Limit	Binding execution allocation (throttling).	Sets quota and period (defaults to 100ms segments).	Java uses limits.

Perspectives on Resource Awareness

It is vital to understand that different layers of your infrastructure interpret these settings differently:

Layer	Behavior
OpenShift/Kubernetes Layer	The scheduler uses requests to manage cluster density and node capacity. It ignores limits for scheduling decisions.
Kernel Layer	The kernel enforces limits via Control Groups (cgroups). Quotas and periods derived from the container spec are translated into filesystem values in `/sys/fs/cgroup`.

Layer	Behavior
Java Layer (OpenJDK)	Current versions of OpenJDK (post-8u191) are container-aware. However, Java only considers limits, not requests. Even if you define CPU requests in your YAML, the JVM will ignore them for internal heap sizing and thread pool calculations.

> ❗ While Java workloads are burstable (threads dynamically demand kernel time), the JVM performs its cgroup reading at **startup** only once. Consequently, the JVM is inelastic, i.e. changing a container limits dynamically and expect the JVM to change its settings without restart. If the user changes limits at runtime without restarting the Pod, the JVM will **not** automatically adjust its internal heuristics to match the new constraints.

Best Practices for OCP Configurations

- **Avoid Overcommitting Critical Workloads:** For production-critical applications, consider Guaranteed Quality of Service (QoS) - this will put the application as the last to be evicted. A pod is considered Guaranteed when both requests and limits are set to the same value for all containers in the pod.
- **Avoid Undercommiting Workloads:** Benchmark the application to use the correct heap percentage (MaxRAMPercentage) to make sure heap and off-heap have space to grow accordingly.
- **Limit vs. Request Parity:** If you are unsure of the application’s baseline, start by setting requests lower than limits to allow for burstable performance, but ensure the limit is high enough to accommodate peak load to prevent CPU throttling.
- **Memory Awareness:** Prefer using memory limits into containers to prevent a single Java container from consuming a large chunck of available node memory and CPU, which can impact other pods on the same node. Unbounded deployments can be very useful, but need to be used in specific scenarios.

Special use cases

Unbounded deployment

Although not the core of this course, it is totally possible and sometimes even desired to have Java applications without container limits, the so called unbouded. Scenarios such as very few java applications, large OCP nodes, and specific scearios for high spike demand can justify its utilization.

This is achieved by creating an deployment without limits, which will have two main consequences:

- Java will use the full host as limit: which can make the application use a very large number of CPUs as baseline (for thread pools) and a large memory utilization (given heap will be based on the host value.
- QoS will be Burstable or Best Effort, depending if requests are set.

This type of application deployment should be treated with caution, given the fact that several applications (unbouded) will cause a race for the hosts resources and may impact directly on CPU throttling (many threads competing at the same time for CPU and memory resources). Consequently, this can overload the host and create more possibilites for eviction scenarios (several Java unbounded demanding more and more resources) causing generalized evictions.

Table 3. Pros and Cons of Unbounded Applications

Pros	Cons
• Use the full Host CPU and Memory • Can spike the full host capability	• Have BestEffort QoS • Competing with other applications

Deployment with 1 CPU

A deployment with one CPU is especial because Java uses CPU for its threads calculations, and the number of threads in Java determine its performance. Any, and all, thread pools that are CPU bounded will be calculated using CPU as 1, and therefore will be significant performance impact on deployments. That is very relevant exactly for GC threads, because with 1 CPU, the SerialGC collector is very much likely - if not certainly - the best performance GC cleaning, because there is no context switch. Specifically for GC, the number of CPUs sets the Parallel threads (for all Parallel phased collectors starting with ParallelGC).

This does not seem much, but it is a core issue if you factor in: - that Java allows using a different GC collector even with 1 CPU and doesn’t overwrite the user setting - run-java.sh sets ParallelGC by default at start up

Therefore, as corrolary of the above, we see that the more performant the application, the more likely more than 1 CPU needs to be used: for CPU bounded pools and for GC performance.

Over all 1 CPU could be fully valid use case for a small deployments such as quarkus microservices, and small springboot applications. However, the reality is much more nuanced than just a table (use case → CPU) and requires benchmarking as means to find the most adequate CPU allocation.

As everything we suggest fully testing and for critical deployments perhaps 1 CPU may not be entirely adequate, again because CPU is used for thread calculations, including Parallel and concurrent threads for GC collectors, and it is used by the kernel for allocating the time for those threads to run.

Default Red Hat OpenJDK’s ENTRYPOINT vs direct Java execution

The above example (hotrod deployment) has 80% as default values, because it has run-java.sh as ENTRYPOINT:

+

```
$ jcmd 1 VM.info | head
1:
#
# JRE version: OpenJDK Runtime Environment (Red_Hat-17.0.20.1.1-1) (17.0.20.1+1)
(build 17.0.20.1+1-LTS)
```

```
# Java VM: OpenJDK 64-Bit Server VM (Red_Hat-17.0.20.1.1-1) (17.0.20.1+1-LTS, mixed
mode, sharing, tiered, compressed oops, compressed class ptrs, parallel gc, linux-
amd64)

---------------  S U M M A R Y ------------

Command Line: -XX:MaxRAMPercentage=80.0 -XX:+UseParallelGC -XX:MinHeapFreeRatio=10
-XX:MaxHeapFreeRatio=20 -XX:GCTimeRatio=4 -XX:AdaptiveSizePolicyWeight=90
-XX:+ExitOnOutOfMemoryError -XX:MaxRAMPercentage=50.0 /deployments/hotrodspringboot-
0.0.1-SNAPSHOT.jar

Host: Intel(R) Xeon(R) Platinum 8370C CPU @ 2.80GHz, 8 cores, 2G, Red Hat Enterprise
Linux release 9.8 (Plow)
---------------  P R O C E S S  ---------------

Heap address: 0x00000000c0000000, size: 1024 MB, Compressed Oops mode: 32-bit
```

Above shows 50% (MaxRAMPercentage at 50%) of 2Gi, which is 1024Mb.

However, observe Java’s internal detection of these limits - executing directly java command (25% expected):

```
java -XshowSettings:vm -version <------------ direct execution inside a container
```

Example output:

```
VM settings:
    Max. Heap Size (Estimated): 123.75M
    Using VM: OpenJDK 64-Bit Server VM

openjdk version "25.0.4.1" 2026-08-18
OpenJDK Runtime Environment (Red_Hat-25.0.4.1.1-0) (build 25.0.4.1+1)
OpenJDK 64-Bit Server VM (Red_Hat-25.0.4.1.1-0) (build 25.0.4.1+1, mixed mode,
sharing)
```

The above is expected: Java is not using the `java-run.sh` ENTRYPOINT and as explained on this course - the MaxRAM percentage at 80% comes specifically from the `run-java.sh` ENTRYPOINT - s2i with OpenJDK injects that `ENTRYPOINT` by default. Without this entrypoint, JAVA_OPTS environment variables are useless.

In the above example, i.e. java straight execution, it will bring 25% because it is the default upstream limit. In other words, limit is set to 512Mi but the heap size displayed is 123.75M when executing java directly inside a Red Hat OpenJDK container.

Understanding cgroups implications for Java applications

In the context of modern container orchestration, the Linux kernel's **control groups (cgroups)** feature acts as the fundamental mechanism for resource isolation. For Java applications running in OpenShift or Kubernetes, cgroups provide the enforcement boundaries for CPU and memory usage.

Understanding how the Java Virtual Machine (JVM) interacts with these cgroups is critical for preventing performance degradation and application crashes.

The Role of cgroups in Resource Management

Cgroups allow the kernel to organize processes into hierarchical groups and allocate resources—such as CPU time, system memory, and network bandwidth—to these groups. When you define `limits` and `requests` in your OpenShift Deployment configuration, OpenShift translates these values into cgroups filesystem entries (typically found under `/sys/fs/cgroup/`). Details:

- cgroups v1: /sys/fs/cgroup/memory/memory.max_usage_in_bytes
- cgroups v2: /sys/fs/cgroup/memory.max

Java developers do not (necessarily) need to understanding specific details on this matter, as long use the latest versions that are fully container-aware and understand that Java detects the container limits.

CPU and Memory Enforcement

- **CPU Limits:** Java applications are inelastic regarding CPU detection - it won't detect every 10s, it will detect once at start-up. Upon startup, the JVM queries the cgroup filesystem to determine how many CPU cores are available. It uses this information to size its internal thread pools (e.g., the Common ForkJoinPool). Once started, this value generally remains fixed. If a container is restricted via a quota, the kernel throttles the CPU cycles available to the JVM.
- **Memory Limits:** The JVM monitors the `memory.limit_in_bytes` (cgroups v1) or `memory.max` (cgroups v2) files. If the JVM is container-aware, it uses these values to calculate default heap sizing. The exact percentage will depend on the version and method of starting: 25% for deployment without run-java.sh, 80% for run-java.sh script (current Red Hat OpenJDK images) and 50% for run-java.sh script started (old Red Hat OpenJDK images).

The Danger of Decoupling

The most significant risk for Java applications in containers is the **decoupling of memory settings** from container limits. This occurs when the JVM is either non-container-aware (older versions) or is manually configured with fixed values that ignore the container's environment.

When the JVM heap size (`-Xmx`) is set higher than the actual memory limit enforced by the cgroup:

1. **The Invisible Limit:** The JVM believes it has more memory available than the container actually provides.

2. **Kernel Intervention:** When the application attempts to allocate memory beyond the cgroup limit, it does not trigger a standard `java.lang.OutOfMemoryError`. Instead, the **cgroup OOM Killer** invokes the kernel to terminate the process immediately.
3. **Observability Gaps:** Because the kernel performs the termination, these events often do not appear as standard Java application exceptions or OpenShift deployment events. This leads to silent failures, node-level instability, or a crash-looping pod that provides little diagnostic information.

Best Practices for cgroup-Safe Java

To ensure your Java applications coexist healthily with cgroup limits, adopt the following strategies:

- **Utilize Container-Aware JVMs:** Always latest versions (or minor versions) Java: Java 8u372+, Java 11, 17, 21+, which are fully container aware natively. These versions automatically read the cgroup filesystem to adjust memory ergonomics on both cgroups v1 and v2.
- **Use Relative Sizing:** Instead of hardcoding `-Xmx` values (e.g., `-Xmx2g`), use container-aware flags like `-XX:MaxRAMPercentage`. This allows the JVM to scale its heap dynamically based on the memory limit assigned to the container (e.g., `-XX:MaxRAMPercentage=75.0` will set the heap to 75% of the container limit).
- **Account for Off-Heap Memory:** Remember that the total memory used by a container includes the heap, the Metaspace, stack space for threads, code cache, and native buffers (Direct Byte Buffers). Ensure the percentage (MaxRAMPercentage) leaves sufficient room for heap to grow and for non-heap segments, or the container will exceed its cgroup limit triggering an OOMKill.

> When debugging, always check the `memory.stat` file inside the container (if permissions allow) or use tools like `jcmd` (VM.info) to inspect current memory usage and compare them against the deployment's defined resource limits.

Navigating Kubernetes resource management for Java

In Kubernetes and OpenShift environments, resource management is governed by the orchestration layer: kubeAPI, which creates deployments with CPU settings. In case this fails (at container creation or deployment) events will show the cause.

- The specific cgroups boundaries are maintained by the Host kernel. In case this fail (memory grows too much) the kernel will trigger an OOMKill event.
- For Java applications, the primary challenge lies in the abstraction gap: the JVM manages its own memory (heap, code cache, metaspace) and CPU (GC threads, JIT compiler threads). In case this fails, the JVM will cause OOExceptions or Errors (crashes).
- Kubernetes manages the container's total resource footprint and keeps the overall applications running nominally via probes, which in case they fail cause SIGTERMs.

Resource Requests vs. Limits

Effective resource management requires a clear understanding of the difference between requests and limits:

- **Requests:** The guaranteed amount of resources (CPU/Memory) allocated to the container. The Kubernetes scheduler uses these values to place pods on nodes.
- **Limits:** The maximum threshold allowed for the container. If a Java application exceeds its memory limit, the Linux kernel OOM (Out of Memory) Killer will terminate the process.

Setting `Limits` equal to `Requests` results in a **Guaranteed Quality of Service (QoS)** class. While this provides the highest priority, it does not prevent the OOM Killer from terminating the pod if the JVM heap plus off-heap memory usage exceeds the container limit.

Cgroups and the Java JVM

Since the introduction of JDK 8u191 (and the backport of `JDK8146115`), the JVM has become container-aware. It reads cgroup filesystem metrics to determine available resources rather than querying the host OS kernel.

When the JVM starts, it calculates heap ergonomics based on the container's cgroup limits:

- **Memory:** If no `-Xmx` is explicitly set, the JVM defaults to a fraction of the container's memory limit - see details on the percentage below.
- **CPU:** The JVM detects the number of available CPUs via cgroup shares/quotas and scales the number of Garbage Collection (GC) threads accordingly.

The Impact of Dynamic Resizing

While recent Kubernetes versions (1.27+) and OpenShift 4.4+ introduced features to resize pod resources without a restart, the JVM behavior remains tied to its initialization state.

1. **Startup Snapshot:** The JVM performs its initial heap and thread pool calculations based on the resource values present when the process is invoked.
2. **Detection vs. Adaptation:** While the JVM can technically observe changes in the cgroup filesystem during runtime, it does not resize its internal heap boundaries (Xmx) dynamically.
3. **Risk:** If a container is vertically scaled (e.g., memory limit increased), the JVM will likely continue to respect the original heap size calculated at boot, potentially leaving the application under-provisioned despite the increase in available container resources.

Best Practices for Java on Kubernetes

To ensure stability in a Kubernetes-managed environment, adhere to the following configurations:

- **Explicit Heap Definition:** Even with container awareness, Java is already contianer aware and this ensures the heap scales proportionally if the container limit is adjusted in the deployment manifest. The best practice is for the user to tune this value (e.g., `-XX:MaxRAMPercentage=75.0`), rather than setting directly Xmx, which overwites anything else.
- **Account for Off-Heap Memory:** Remember that the total container memory limit must accommodate the Java heap, Metaspace, Code Cache, Thread stacks, and Native memory (NMT). Tune `-XX:MaxRAMPercentage` conservatively (e.g., 50-75%) to prevent OOM Exceptions events at

heap level.

- **CPU Throttling:** Be aware that setting strict CPU limits can lead to CPU throttling, where the kernel pauses threads when the quota is reached. Monitor `container_cpu_cfs_throttled_periods_total` in Prometheus to identify performance bottlenecks. The application's threads will be paused during this throttling process, which will show as a plateau in Prometheus CPU usage.
- **Adequate CPU settings:** Lack of memory will cause OOME but lack cpu will not cause a crash, but instead the threads will take so long to execute (given the kernel will give so little kernel time for the threads).

> Always use the latest stable version of Red Hat's OpenJDK, as it includes the most robust cgroup v2 support and the latest optimizations for container environment interactions.

Troubleshooting and Tooling

Troubleshooting Java in containerized environments requires distinguishing between tools that report host-level node metrics versus those that are container-aware. Improper resource utilization undoubtedly will lead to memory misconfiguration, `OOMKills`, OOMExceptions, SIGTERMs, crashes (hs_error crashes), and under/over-utilization of resources.

- **Host-level vs. Container-aware Tools**: Standard Linux utilities like `lscpu`, `nproc`, and `free -m` reflect the underlying node's resources rather than container limits.
- **Container-aware Limits**: Utilize JVM-specific tools such as jcmd's `VM.info` or `-XX:+PrintFlagsFinal` (via `--XshowSettings:system`) to accurately retrieve container memory and CPU constraints enforced on the container.
- **Container-aware Diagnostics**: Utilize JVM-specific tools such as jcmd's `VM.info` to accurately verify the JVM's memory (percentage), thread details, and CPU constraints currently being used by Java at runtime.
- **Deployment Inspection**: Analyze pod logs to verify if scripts like `run-java.sh` are active or if the default OpenJDK memory allocation is scaling correctly.
- **Heap Analysis**: Compare the JVM's reported heap size against container limit values to identify potential discrepancies between configured `-Xmx` values and actual resource availability.
- **Resource Tuning**: Be aware that manually overriding `-Xmx` on container-aware JVMs can cause mismatches with container limits, potentially leading to container underutilization or runtime `OOMExceptions`.

Identifying Container-Aware versus Non-Container-Aware Tools

When managing Java applications in Red Hat OpenShift or Kubernetes, understanding whether your diagnostic tools perceive the underlying host node or the isolated container environment is critical. Misinterpreting these outputs often leads to OutOfMemory (OOM) kills or CPU throttling because the application may be relying on node-level visibility rather than container-level bondaries.

Distinguishing Tool Behavior

To effectively troubleshoot Java in containers, you must categorize your diagnostic tools based on their ability to parse `cgroups` (control groups).

Non-Container-Aware Tools

Non-container-aware tools report information regarding the physical or virtual host (the OCP Node). Relying on these tools within a pod will provide a false sense of resource availability.

- **`lscpu`**: Reports the total CPU architecture and count of the physical worker node, not the specific CPU quota assigned to your pod.
- **`nproc`**: Returns the number of processors available to the host OS, ignoring the Kubernetes

`resources.limits.cpu` defined in your pod specification.

- **`free -m`**: Displays the total RAM available to the node. If your pod has a 512MB limit but the node has 64GB, this command will inaccurately suggest you have 64GB available for your heap.

Container-Aware Tools

Container-aware tools interface with the Linux `cgroups` subsystem to identify the specific constraints imposed by the orchestrator. These are the "source of truth" for your application environment.

- **`-XX:+PrintFlagsFinal`** / **`-XshowSettings:system`**: These flags instruct the JVM to report the memory and CPU settings it has calculated based on container limits. Use these to verify if the JVM correctly identified the container memory boundary.
- **`VM.info` (via `jcmd`)**: Provides a snapshot of the JVM, including flags and internal settings that reflect whether the container-aware logic was successfully applied.
- **`run-java.sh`**: Always present in Red Hat OpenJDK images, sets certain specific Java flags, such as ParallelGC and MaxRAMPercentage Java heap sizes automatically before the JVM starts. In current images, this script won’t make Java container aware (that is on the Java code itself) but it will make a difference on the Heap percentage that comes by default (MaxRAMPercentage) value, in case it is not used the default OpenJDK container should set heap at 25% of the deployment (upstream default value).

Diagnostic Strategy

To verify if your environment is correctly configured, compare the output of host-level tools against JVM-internal tools.

Tool	Scope	Usage Case
`free -m`	Node	Use only to verify the underlying node capacity.
`nproc`	Node	Ignore when calculating thread pools or garbage collection threads.
`java -XshowSettings:system`	Container	Use to confirm the JVM sees the pod’s `limits.memory`.
`jcmd <pid> VM.info`	Container	Use for deep runtime inspection of detected resource limits.

Detecting Misconfiguration

If your application is crashing with an `OOMKiller` signal, follow this diagnostic workflow:

1. **Check JVM visibility**: Run `java -XshowSettings:system` inside the running container.
2. **Compare limits**: Look at `MaxHeapSize` and `TotalPhysicalMemory` in the output.
3. **Evaluate OCP Manifests**: Compare these values against your OpenShift `Deployment` or

`DeploymentConfig` resource requests and limits.

4. **Log Analysis**: Review the pod startup logs. If the logs show that the JVM is using a default (e.g., 25% of physical memory) rather than a percentage of the container limit, the JVM is likely not properly detecting the `cgroup` constraints.

> If the application logs indicate it is defaulting to 25% of the **node's** memory rather than the **container's** limit: verify the OpenJDK version (8u191+) is correctly detecting containers limits, e.g. for cgroups v2 compliance. If the application logs indicate it is defaulting to 25% of the **container's** memory rather than t50% or 80%: verify if the application is deployed with run-java.sh as ENTRYPOINT (that's the default).

Analyzing Container Limits and Resource Utilization

To ensure Java applications perform predictably within Red Hat OpenShift or Kubernetes, you must understand how the JVM interacts with the underlying control groups (cgroups). When an application is container-aware, the JVM can query the cgroup filesystem to determine its actual boundaries rather than defaulting to the physical host's resources.

Understanding Resource Visibility

The primary challenge in containerized Java is the discrepancy between what the Operating System sees (physical host) and what the container runtime enforces (cgroup limits).

The Mechanics of Detection

Container-aware JVMs (starting from Java 8u191 and newer) automatically inspect cgroups files hierarchy, such as `memory.limit_in_bytes` to size the heap, but cgroups will also be used to determine the available processor count that are used for Java thread pool calculations. Latest Java versions are fully cgroups v1 and v2 complaint.

When analyzing your application, you can extract this metadata using `jcmd`. If your JVM is properly detecting the container environment, the output will reveal specific cgroup configurations:

```
# Example output from jcmd <pid> VM.check_commercial_features or custom diagnostic
commands
container_type: cgroupv1
active_processor_count: 1
cpu_quota: -1
memory_limit_in_bytes: 536870912
memory_usage_in_bytes: 125108224
```

- `active_processor_count`: Essential for the JVM to size its internal thread pools (e.g., ForkJoinPool).
- `memory_limit_in_bytes`: Dictates the physical memory limit the JVM uses to calculate its heap size. By default the percentage for heap max size will be 25% (on upstream default) or 80%

(when starting with `run-java.sh`).

Identifying Limitations and Misconfigurations

Analyzing resource utilization requires verifying that your JVM heap plus off-heap requirements remain within the Kubernetes/OpenShift `limits`.

The Decoupling Risk

There are several risks of decoupling, i.e. manually setting Xmx separately from the container size:

- The Xmx set the full size of the heap, given it is unrelated so the container size can reduce and starve the application or have too much space useless.
- A common pitfall is ignoring the off-heap footprint. If the Java Heap (Xmx) to 80% of your container limit, but your application requires 30% for off-heap memory (Metaspace, Code Cache, Thread stacks, Direct Buffers), the container will exceed its memory limit and OOMKill.

This results in a **cgroup OOMKill**, which is distinct from a JVM `java.lang.OutOfMemoryError`.

Troubleshooting Checklist

When analyzing resource constraints, follow these steps:

1. **Compare Requests vs. Limits:** Ensure your OpenShift `requests` are sufficient for startup, while `limits` accommodate peak load.
2. **Verify Cgroup Awareness:** Use `jcmd` to verify if the JVM identifies the correct `memory_limit_in_bytes`. If this shows `-1` or reflects host memory, the JVM is not container-aware, and you must explicitly configure `-XX:MaxRAMPercentage`.
3. **Monitor via Tools:**
    - **Container-aware:** Use `kubectl top pod` or the OpenShift Web Console metrics.
    - **Non-container-aware:** Using `top` or `free` commands inside the container will report host-level data rather than container-constrained data.

Troubleshooting table

Event / Characteristic	Source & Trigger	Impact & Log Signature
OOMKilled (Linux Kernel / cgroups)	Triggered by the operating system when the container's total memory footprint (Heap + Native Memory) exceeds the cgroup hard limit.	**Impact:** The kernel abruptly terminates the process (SIGKILL, exit code 137). Container exits. **Log Signature:**

```
[16840841.104477] Killed
process 117061 (java), UID
1000... anon-rss:6178796kB
```

Event / Characteristic	Source & Trigger	Impact & Log Signature
OutOfMemoryError (Java / JVM)	Triggered by the JVM when it exhausts internally configured memory pools (e.g., Heap or Metaspace), even if the container has physical memory left.	**Impact:** Java throws an exception. Container may stay alive in a broken state unless `-XX:+ExitOnOutOfMemoryError` is used. **Log Signature:** `java.lang.OutOfMemoryError: Java heap space`
SIGTERM (Graceful Shutdown)	Triggered by OpenShift/Kubernetes when scaling down, deploying a new version, or deleting a Pod.	**Impact:** The JVM receives Signal 15 and triggers shutdown hooks. If it takes longer than `terminationGracePeriodSeconds` (default 30s), it is force-killed (SIGKILL). Container exits gracefully (often exit code 143). **Log Signature:** `INFO: JVM running shutdown hooks...`

JVM Crash (`hs_err_pid` file)	Triggered by a segmentation fault (SIGSEGV), native library bug (JNI), or internal JVM bug. It is a failure at the C/C++ level of the runtime.	**Impact:** The JVM aborts immediately without running shutdown hooks and dumps a crash log (`hs_err_pid<pid>.log`) to the working directory. Container exits. **Log Signature:**
		`# A fatal error has been detected by the Java Runtime Environment:`
		`#  SIGSEGV (0xb) at pc=0x00007f3a8b411c52...`

References / Further Reading

- For more details on this behavior, refer to the Red Hat Knowledgebase article: Troubleshoot options for Data Grid pod crash

Verifying JVM Resource Awareness (Reference)

Use the following as general guidance to confirm that a running pod is correctly interpreting its cgroup limits. For the full hands-on Native Memory Tracking (NMT) walkthrough — enabling `-XX:NativeMemoryTracking` and reading `jcmd 1 VM.native_memory summary` — follow Analyzing Heap and Off-Heap Settings.

- **Access the pod and identify the Java PID** with `oc rsh <pod-name>` followed by `jps` (or `ps -aux`, or `jcmd`); the application typically runs as PID 1.
- **Check resource visibility** with `jcmd <pid> VM.native_memory summary` and look for the container-specific configuration in the diagnostic output.
- **Cross-reference with OpenShift limits:** Compare the `memory_limit_in_bytes` reported by the JVM with the memory limit defined in the deployment YAML. If they do not align, verify that the application is container-aware and using `run-java.sh`. When `run-java.sh` is the entrypoint, you can adjust `JAVA_OPTS` and `JAVA_OPTS_APPEND` with `-XX:MaxRAMPercentage`; these environment variables only take effect with that entrypoint.

```
# Example:
$ podman run --memory-reservation=700m --memory=500m --memory-swap=1000m --rm -it
localhost/openjdk:app
memory_limit_in_bytes: 512000 k <------------------- memory 500mb
memory_and_swap_limit_in_bytes: 1024000 k <--------- swap and memory 1000mb
memory_soft_limit_in_bytes: 716800 k <-------------- soft limit 700mb
```

If you observe `cgroup OOMKills` despite the JVM reporting correct memory percentage (e.g. 25%, 50%, or 80%), analyze your off-heap usage via `jcmd <pid> VM.native_memory summary`. You may need to decrease your `MaxRAMPercentage` to provide more breathing room for the native memory footprint, such as instrumentation or `Metaspace`.

Diagnostic strategies for Java deployments in OpenShift

When managing Java applications in OpenShift, traditional diagnostic tools often fail because they lack container-awareness capabilities. Tools that look at the host operating system’s total resources rather than the container’s constrained cgroup limits can lead to misleading metrics. Effective diagnostics require tools and strategies that respect the isolation boundaries defined by Kubernetes.

Identifying Container-Aware Tools

A critical diagnostic step is distinguishing between container-aware and non-container aware tools.

- **Non-container aware Tools:** Tools like `top`, `free`, or older JVM monitoring utilities may report the host’s total RAM and CPU count. Relying on these leads to "false positives" where an administrator believes the application has more resources than it actually does.
- **Container-Aware Tools:** Modern JDKs (8u191+, 11+) and utilities like `jcmd`, `jstat`, and modern observability agents are cgroup-aware. They query the container’s `/sys/fs/cgroup/` filesystem to report limits specifically applied to the pod.

Diagnostic Strategies

To effectively diagnose resource constraints or performance issues, follow these strategies:

1. Validating Resource Visibility

Verify that the JVM sees the limits defined in your OpenShift `DeploymentConfig` or `Deployment`. You can inspect this by executing into the pod and checking the container's perceived memory and CPU:

```
# Check perceived heap from within the pod
$ oc exec hotrod-app-ID -- jcmd 1 VM.info | grep  "Heap address"
Heap address: 0x00000000c0000000, size: 1024 MB, Compressed Oops mode: 32-bit
```

Verify the JVM limits being detected:

```
# Check perceived limit from within the pod
oc exec <pod-name> -- jcmd 1 VM.info | grep -A18 "container (cgroup) information:"
```

For the full annotated `container (cgroup) information` output and how each field maps to your YAML limits and requests, see Configuring Limits and Requests in Red Hat OpenShift.

Check if Heap is calculated correctly based on the container memory limit rather than the worker node's total memory.

2. Utilizing Java Diagnostics - `jcmd`

Java diagnostics, jcmd, is very useful for Java and Java inside containers in general. It allows you to perform JVM dump, heap dumps, thread dumps, and analyze JVM flags without requiring a restart.

Diagnostic Tool	Purpose / Action
Thread Dumps	Identify if the application is hitting thread limits or experiencing deadlocks.
GC Performance	Use `jcmd <pid> GC.run` to trigger garbage collection or `jcmd <pid> VM.flags` to verify if `-XX:MaxRAMPercentage` is adequately set (too high or too low).
VM.info	Use `jcmd <pid> VM.info` to verify if `-XX:MaxRAMPercentage`, Native allocation, cgroups details and a lot more.

3. Analyzing cgroup Constraints

If a pod is frequently restarting, it may be hitting OOM (Out of Memory) kills. Since the OOM Killer is enforced at the cgroup level, standard Java `OutOfMemoryError` logs might not appear if the kernel terminates the process before the JVM can report it.

- **Inspect Pod Events:** Use `oc describe pod <pod-name>` to check for the `OOMKilled` state.
- **Check Exit Codes:** An exit code of `137` is a strong indicator that the container was terminated by

the OOM killer because it exceeded its respective container memory limit.

4. Usage of JVM flags for scenarios

For specific scenarios, it is possible to use certain flags to verify the issue:

Scenario	Exit Code	When to Use It
`-XX:+HeapDumpOnOutOfMemoryError`	3	Use when you need to diagnose memory leaks. It captures the exact state of the heap into a `.hprof` file at the moment the `OutOfMemoryError` occurs for offline analysis.
`-XX:+ExitOnOutOfMemoryError`	3	Use as a best practice in Kubernetes/OpenShift. It forces the JVM to terminate immediately upon an OOM exception, allowing the kubelet to restart the pod instead of leaving it running in a broken, unresponsive state.
`-XX:+CrashOnOutOfMemoryError`	0	Use for deep native-level debugging. Instead of just exiting, the JVM produces a fatal crash log (`hs_err_pid.log`) and a core dump, bypassing standard shutdown hooks.
`kill -11 <PID>`	0	Use during testing to simulate a native segmentation fault (SIGSEGV). This verifies how your container and OpenShift handle sudden, catastrophic C/C++ level crashes.
Setting `-Xmx`/`-Xms` > container size (plus `-XX:+AlwaysPreTouch`)	137	Use to intentionally trigger an OS-level `OOMKilled` event. This is helpful to test OpenShift resource quota enforcement, eviction behavior, or monitoring and alerting rules.

Analyzing Resource Constraints

Step 1: Verify JVM resource settings

Use the `hotrod-app-*` container from the previous exercise to confirm the JVM's container (cgroup) awareness before diagnosing throttling. The expected heap-to-container ratios (80% / 50% with `run-java.sh`, 25% for direct `java`) are explained in Tuning Java Heap Settings Based on Container Limits.

Step 2: Use `jcmd` to analyze the process flags

Access the container shell to utilize `jcmd`.

```
oc rsh <pod-name>

# List running Java processes
jcmd
```

```
# Get a summary of the current flags the JVM is using
jcmd <pid> VM.flags

# Get a detailed report of the VM's perceived memory constraints
jcmd <pid> VM.info
```

Step 3: Troubleshooting a constraint

If the application is impacted, check if CPU throttling is occurring due to the Kubernetes `cpu` limit/requests vs the utilization on the kernel (kernel is the one throttling, not Kubernetes):

1. Verify first if the Java threads are nominal

   ```
   $ jcmd PID Thread.print <-- see the threads
   ```

2. Then, in case CPU throttling impacting, see the `cpu.stat` file within the container:

   ```
   cat /sys/fs/cgroup/cpu.stat <--------- cgroups v2
   usage_usec 9123943
   user_usec 8159424
   system_usec 964518
   core_sched.force_idle_usec 0
   nr_periods 2125
   nr_throttled 21
   throttled_usec 878173
   nr_bursts 0
   burst_usec 0
   ```

Above shows details for cgroups v2, for cgroups v1 use `/sys/fs/cgroup/cpuacct.usage`. In case the JVM is not cgropus v2 compatible, cgroups won’t be detected `No metrics available for this platform` may be shown.

1. For CPU throttling, verify `nr_throttled`. If this number is increasing rapidly, the CPU `limit` may be too low, and the kernel is throttling your application, causing latency. Therefore tuning the ratio (requests vs limits) can be adequate.

```
$ oc set resources deployment/hotrod-app --limits=cpu="2" --requests=cpu="1"
```

Best Practices Summary

- **Avoid reling on `free` or `top`** inside a container for memory capacity verification.
- **Always use `-XX:MaxRAMPercentage`** instead of fixed `-Xmx` values to ensure the heap scales dynamically with the container limit, avoiding decoupling.
- **Prioritize `jcmd`** for all runtime analysis, as it is native to the OpenJDK and fully supports container-cgroup awareness.

The following JVM flags can be used for troubleshooting:

JVM Flag	Description
`-XX:+ErrorFileToStdout`	Sends the crash output directly to standard output, making it visible in pod logs (`oc logs`).
`-XX:ErrorFile=<path>`	Sets the destination path for the fatal crash log file (`hs_err_pid.log`).
`-XX:HeapDumpPath=<path>`	When used alongside `-XX:+HeapDumpOnOutOfMemoryError`, this sets the directory or exact file path where the heap dump will be written to.
`-XX:+CrashOnOutOfMemoryError`	Crashes the JVM (generating a crash log and core dump) instead of throwing a standard `OutOfMemoryError`.
`-XX:+ExitOnOutOfMemoryError`	Forces the JVM to exit immediately on an out-of-memory exception.
`-XX:OnOutOfMemoryError="<cmd>"`	Defines a specific action (command or script) to execute when an `OutOfMemoryError` happens.
`-XX:OnError="<cmd>"`	Defines a specific action (command or script) to execute when a fatal JVM crash occurs.

Java Diagnostics: jcmd

The `jcmd` utility is a versatile diagnostic tool included with the JDK. In containerized environments, it is indispensable for inspecting the internal state of a running Java Virtual Machine (JVM) without requiring external profilers or complex agent attachments.

Role of Java Diagnostics (jcmd) in Containerized Environments

In a containerized world, `jcmd` allows developers and operators to bridge the gap between the container's resource limits (cgroups) and the JVM's internal resource allocation. When troubleshooting performance or memory issues in OpenShift or Kubernetes, `jcmd` provides a direct view into the JVM's view of the container.

Key Capabilities

- **Process Identification:** List running Java processes in the container owned by the user running jcmd.
- **Performance Diagnostics:** Retrieve thread dumps, GC statistics, and class loading metadata.
- **Memory Management Analysis:** Inspect heap usage and native memory tracking (NMT).
- **Dynamic Configuration:** Adjust certain JVM flags at runtime (when the flag is adjustable).

Using jcmd for Resource Inspection

The main tool for Java investigations inside a container is `VM.info`, which is only avaliable in JDK 11+. It prints several details on the Java process itself, from the JVM perspective. See full review on

Interpreting VM.info file in OpenJDK/OracleJDK.

When a container is suffering from `OutOfMemory` (OOM) errors or unexpected latency, you can execute `jcmd` directly inside the containerf to gather data.

```
# List all running Java processes
jcmd

# Java full dump  (JDK 11+)
jcmd <PID> VM.info

# Retrieve a thread dump for a specific PID
jcmd <PID> Thread.print

# Check GC performance statistics
jcmd <PID> GC.run
```

Analyzing Native Memory

One important aspects of Java inside a container is the memory allocation in **Java Heap** and **Off-Heap memory**. If the container is being killed by the OOMKiller but the heap appears stable and healthy, the issue may reside in native memory (e.g. libraries, Metaspace, code cache, or direct buffers).

Dynamic memory tracking

For tracking Dynamic memory allocation see `Dynamic Libraries` section in `VM.info`:

```
$ oc exec hotrod-app-ID -- jcmd 1 VM.info
...
...
Dynamic libraries:
c0000000-c1580000 rw-p 00000000 00:00 0
c1580000-eab00000 ---p 00000000 00:00 0
eab00000-eb500000 rw-p 00000000 00:00 0
eb500000-100000000 ---p 00000000 00:00 0
56381ca00000-56381ca01000 r-xp 00000000 08:24 281334450
/usr/lib/jvm/java-17-openjdk-17.0.20.1.1-1.2.el9.x86_64/bin/java
56381cc01000-56381cc02000 r--p 00001000 08:24 281334450
/usr/lib/jvm/java-17-openjdk-17.0.20.1.1-1.2.el9.x86_64/bin/java
56381cc02000-56381cc03000 rw-p 00002000 08:24 281334450
/usr/lib/jvm/java-17-openjdk-17.0.20.1.1-1.2.el9.x86_64/bin/java
...
7f1e89b37000-7f1e89b39000 r-xp 00000000 08:24 190036142
/usr/lib/jvm/java-17-openjdk-17.0.20.1.1-1.2.el9.x86_64/lib/libextnet.so
7f1e89b39000-7f1e89d38000 ---p 00002000 08:24 190036142
/usr/lib/jvm/java-17-openjdk-17.0.20.1.1-1.2.el9.x86_64/lib/libextnet.so
7f1e89d38000-7f1e89d39000 r--p 00001000 08:24 190036142
```

```
/usr/lib/jvm/java-17-openjdk-17.0.20.1.1-1.2.el9.x86_64/lib/libextnet.so
```

Native Memory Tracking

Enable Native Memory Tracking (NMT) via the JVM argument `-XX:NativeMemoryTracking=summary` to use `jcmd` is useful to track native memory allocation.

```
# View native memory summary
jcmd <PID> VM.native_memory summary
```

Best Practices for Diagnostics in OpenShift

1. **Accessing the Pod:** Use `oc rsh <pod-name>` to gain shell access to the running container, otherwise, use `exec` to have the output executed outside the container.
2. **Tooling Availability:** Ensure your container base image includes the `jcmd` binary (standard in most OpenJDK-based images).
3. **Non-intrusive Data Collection:** Use `jcmd` for point-in-time snapshots. Avoid high-frequency polling, as diagnostics can consume CPU cycles within the container's limited quota.
4. **Verification:** Compare the output of `GC.heap_info` against the container's `resources.limits.memory` to verify if the JVM is respecting the container's memory boundaries.

Inspecting JVM Resources

In this exercise, we will verify the JVM's memory awareness.

1. **Access the container terminal:**

   ```
   oc rsh deployment/my-java-app
   ```

2. **Identify the PID of the Java process:**

   ```
   $ jcmd
   ```

3. **Inspect the current heap configuration:** Use the following command to check how the JVM is interpreting the container memory limits:

   ```
   $ jcmd <PID> VM.flags
   ```

   *Look for `Heap Address` line, section `size` and `GC Precious Log`, section `Heap Max Capacity`.

4. **Analyze GC performance:** Check if the Garbage Collector is struggling due to container constraints:

```
$ jcmd <PID> GC.class_histogram
```

Troubleshooting Tip: If `jcmd` reports a heap size significantly disassociated from the container memory limits, verify:

- re-deploy the application to make sure the value is dynamic.
- check your deployment YAML to ensure you are not manually overriding the heap size with `-Xmx` at runtime
- check your build details if the containerfile/build details itself is not overriding the heap size at build time
- check if the Java is adequately detecting cgroups (8u372+, 11.0.16+, 17+ for cgroups v2) or has an initialization script that overwrites the default values.

Tuning, Monitoring, and Troubleshooting Container-Aware Java

This capstone module puts the container-awareness principles from the earlier chapters into practice on a running OpenShift deployment. Rather than repeating the core CPU and heap walkthroughs, each lab focuses on its own distinct skill and links back to a reference exercise for the shared setup and verification steps:

- **Resource Tuning:** Move from fixed `-Xmx` to percentage-based heap flags (`MaxRAMPercentage`, `InitialRAMPercentage`, `MinRAMPercentage`), account for off-heap memory, avoid baking memory settings into the image, and tune the heap percentage per entrypoint (current `run-java.sh` defaults to 80% whereas upstream deploys is 25%).
- **Monitoring:** Verify the distinction container-aware output (`jcmd VM.info`, `java -XshowSettings:system`) from non-container aware tools (`free`, `nproc`, `lscpu`, `top`), and read the `container (cgroup) information` section to confirm the JVM sees the pod's limits.
- **Troubleshooting:** Correlate the OpenShift, kernel/cgroup, and JVM layers using `oc describe`, `oc debug`, and `jcmd`, then deliberately trigger and diagnose a `cgroup` OOMKill — reading pod information, exit code 137, and the terminated reason — to connect misconfigured limits to an observable failure.

Tuning Java Heap Settings Based on Container Limits

In containerized environments orchestrators, Red Hat OpenShift, Red Hat OpenJDK is able to detect the container boundaries. Instead, it respects the `cgroup` limits assigned to the container. Tuning the Java heap using percentages is the recommended and standard approach to ensure the JVM remains flexible and associated with the container's resource allocation.

The Shift to Percentage-Based Memory Management

Historically since Java started, developers used the `-Xmx` flag to set directly the heap size as absolute value (e.g., `-Xmx2g`). However, deploying with container aware Java inside a container, Java will calculate (entirely by default - user needs to do nothing) the Java heap size as a percentage from the container size. Therefore, setting `Xmx` manually becomes a liability making Java ignore the actual memory limit of the container. If the heap is inadequately set the JVM will eventually exceed the container size leading to a `cgroup` OOM-Kill.

The default behavior uses **RAM Percentage flags** where by default the heap size is associated with the container's memory limit, allowing the JVM to calculate its heap size dynamically based on the container's available memory and re-size upon JVM restart.

Key Configuration Flags

- `-XX:MaxRAMPercentage=N`: Sets the maximum heap size as a percentage of the container's memory limit. The default upstream limit is25%, though specialized container images (like Red Hat's `run-java.sh` scripts) will default to 80%.
- `-XX:InitialRAMPercentage=N`: Sets the initial heap size as a percentage of the container limit.

- `-XX:MinRAMPercentage=N`: Defines the minimum heap size used when the container memory limit is small.

When tuning these percentages, remember that the **total container memory** must accommodate more than just the heap. One must account for **off-heap memory**, which includes:

- Metaspace (Class metadata)
- Thread stacks
- Code Cache
- GC structures and internal JVM native memory
- Off-heap buffers (used by Netty, AMQ, or Data Grid)

Setting `-XX:MaxRAMPercentage` too high leaves insufficient room for native memory, inevitably leading to native memory pressure and `cgroups` OOM-Killer. The exact percentage cannot be precised without application direct knowledge. Details on Can Red Hat provide optimal MaxRAMPercentage or related flags for SpringBoot or Java applications.

Tuning Percentage-Based Heap Settings

This section covers the configuration the user can tune container-aware memory settings instead of hardcoded heap limits.

For the full hands-on walkthrough — changing `MaxRAMPercentage` from 80% to 50% on the running `hotrod-app`, then verifying the resulting `Heap Max Capacity` with `jcmd 1 VM.info` — follow Analyzing Heap and Off-Heap Settings. That lab reuses the deployment from Inspecting CPU-Aware Deployments in OpenShift.

Inspect the current configuration

First, verify that your application is running without explicit `-Xmx` flags or is already using percentage-based flags:

```
# Check the deployment environment
oc set env deployment/<deployment-name> --list
```

Do **not** set the memory constraints into the image, in fact all JVM flags should be set at runtime rather than build time. The following Containerfile (Dockerfile) pattern should be avoided — set the percentage through deployment environment variables instead, so it stays decoupled from the image:

```
FROM registry.access.redhat.com/ubi9/openjdk-17:latest

# Copy the compiled JAR into the standard deployments folder
COPY target/hotrod-app.jar /deployments/hotrod-app.jar
```

```
# Pass memory constraints using JAVA_OPTS_APPEND (parsed by run-java.sh) - DO NOT DO
THIS:
ENV JAVA_OPTS_APPEND="-XX:MaxRAMPercentage=50.0"

# No ENTRYPOINT required; the base image automatically executes run-java.sh
```

Default heap allocation by entrypoint

By default an OpenJDK application is container-aware and will detect the container and size the heap from there. The difference is the percentage:

Execution Method	Default Heap Allocation
Red Hat OpenJDK's run-java.sh ENTRYPOINT	80% (previously was 50%)
Direct java ENTRYPOINT	25%

Tuning the percentage

The default comes for free, so the user has to do nothing to set it up. To tune it, apply the environment variable that matches your entrypoint:

Entrypoint	Environment Variable Configuration
run-java.sh ENTRYPOINT	`$ oc set env deployment/hotrod-app JAVA_OPTS_APPEND="-XX:MaxRAMPercentage=50.0"`
java APP ENTRYPOINT	`JAVA_TOOLS_OPTIONS="-XX:MaxRAMPercentage=75.0 -XX:InitialRAMPercentage=50.0"`

After the deployment restarts, validate that the JVM calculated the heap size relative to the pod's memory limit (look for the active `-XX:MaxRAMPercentage` flag and the resulting `VM.max_heap_size`) by following the `jcmd` verification in Analyzing Heap and Off-Heap Settings. Then monitor memory usage via the OpenShift web console or CLI, keeping usage below the container limit to leave headroom for native off-heap memory.

If you observe `java.lang.OutOfMemoryError: Metaspace` verify the MaxMetaspace flag, which sets the Metaspace limit and although present in the off-heap has its own cap. Details Metaspace setting and tuning in JDK 8+ Applications: in and outside containers.

Reference Documentation

For complete details and subscriber-exclusive guidance on analyzing these diagnostic artifacts, refer to the official Red Hat knowledgebase article:

- Red Hat Solution 6973111: Interpreting VM.info file in OpenJDK/OracleJDK

Common Properties

The `VM.info` file will include:

- JVM build details and version strings
- Operating system environment specifications
- Ergonomics and default sizing parameters

Monitoring CPU and Memory Usage with Container-Aware Tools

When deploying Java applications in containerized environments like Red Hat OpenShift or Kubernetes, traditional Linux diagnostic tools often fall short. Tools that are not container-aware report the physical or virtual resources of the entire host node rather than the specific constraints applied to your pod.

This section explores how to distinguish between container-aware and non-container-aware outputs and how to use them effectively for troubleshooting.

Container-awareness tools vs Non-container aware

If you use standard Linux utilities such as `lscpu`, `nproc`, or `free -m` inside a container, you are likely seeing the resources of the underlying host node. Relying on these values for capacity planning and testing likely will lead to OOMExceptions (OOMException), Out of Memory (OOM) kills or CPU throttling because the JVM may attempt to allocate resources exceeding the container's cgroup limits.

To monitor your Java application accurately, you must utilize tools and flags that interface directly with the **cgroups** filesystem.

Container-Aware Diagnostic Tools

Usage of Java Diagnostics (jcmd) is always preffered.

1. JVM Flags:

The JVM provides built-in mechanisms to report exactly what it sees regarding container limits.

- **`--XshowSettings:system`**: This is the most critical flag for initial diagnosis. It displays the system information the JVM has detected, including cgroup-enforced memory limits and the active processor count.
- **`VM.info`**: When using `jcmd`, this command provides a comprehensive dump of the current JVM configuration, including container-specific constraints.

2. Interpreting Container-Aware Output

When running a container-aware command (such as jcmd's VM.info's and `java -XshowSettings:system -version`), look for the following key indicators:

- `active_processor_count`: Represents the number of CPUs available to the JVM based on the Kubernetes/OpenShift CPU limits.
- `memory_limit_in_bytes`: Reflects the specific cgroup memory limit. If this value does not match your expected Kubernetes resource limit, the JVM is not correctly detecting the container boundary.
- **active processor**`: this shows how many CPU were detected.

Example:

```
container (cgroup) information:
container_type: cgroupv2
cpu_cpuset_cpus: 0-3
cpu_memory_nodes: 0
active_processor_count: 3 <---------------- process count
cpu_quota: 220000 <------------------------ 2.2 cpus
cpu_period: 100000
cpu_shares: no shares
memory_limit_in_bytes: 12582912 k <-------- 12gb limit
memory_and_swap_limit_in_bytes: 12582912 k
memory_soft_limit_in_bytes: unlimited
memory_usage_in_bytes: 758948 k
memory_max_usage_in_bytes: 759060 k
kernel_memory_usage_in_bytes: 10624 k
kernel_memory_max_usage_in_bytes: unlimited
kernel_memory_limit_in_bytes: 10628 k
maximum number of tasks: 98938
current number of tasks: 98
```

Verifying Container Limits

The step-by-step procedure for this comparison — logging into the pod and contrasting non-container aware tools (`free -m`, `nproc`) against container-aware output (`java -XshowSettings:system -version` and `jcmd $PID VM.info`) — is walked through on the same `hotrod-app` deployment in Inspecting CPU-Aware Deployments in OpenShift, which contrasts `nproc` with `active_processor_count`.

As you run it there, keep this difference in mind — it is the point of this activity:

- **Non-container-aware tools** (`free -m`, `nproc`, `top`, `lscpu`) report the resources of the entire OpenShift worker node, **not** the container. Do not use them for capacity planning inside a container.
- **Container-aware output** (`java -XshowSettings:system`, `jcmd $PID VM.info`) reflects the cgroup limits actually applied to the container. Confirm that `memory_limit_in_bytes` and `active_processor_count` match the `limits` defined in side the pod YAML; if they differ, verify your JVM is 8u191 or later, as older versions lack proper cgroup awareness. The versions 8u372+, 11.0.16+, 17+ are required for cgroups v2 compliance.

For the full annotated `container (cgroup) information` block and how each field maps to your

YAML, see Configuring Limits and Requests in Red Hat OpenShift.

Troubleshooting Resource Constraints in and out of OpenShift

When Java applications run in OpenShift, user can define container limits within the Java application where the Linux's kernel control groups (cgroups) set memory and/or CPU boundaries. If these constraints are unset, misconfigured or if the Java Virtual Machine (JVM) is unaware of them, one may encounter performance degradation, unexpected `OutOfMemoryError` (OOM) exceptions (that may, or may not exit the container), container restarts due to OOMKills or even generic OOMKills in the full OCP cluster.

This section focuses on diagnostic strategies and using tools to inspect resource constraints for Java workloads in OpenShift.

Diagnostic Strategy

To effectively troubleshoot resource constraints, you must understand three distinct layers:

1. **OpenShift/Kubernetes Layer:** The resource requests and limits defined in your `Deployment` or `Pod` specification.
2. **Kernel/Cgroup Layer:** The actual hardware quotas enforced by the container runtime.
3. **JVM Layer:** The memory and CPU settings perceived by the Java process. Java only cares about limits for Java's thread setting.

Identifying Misalignment

A common pitfall is the misalignment between the container limit and the JVM heap size. If your container limit is 1Gi but the JVM is configured with `-Xmx2G` (which should be avoided), the process will likely fail immediately or fail shortly after starting, being killed by the kernel when it attempts to allocate memory beyond the cgroup boundary.

Tools for verification

Using `oc describe`

The first step in any troubleshooting process is verifying the resource configuration of the pod.

```
oc describe pod <pod-name>
```

- **Look for:** The `Limits` and `Requests` sections.
- **Action:** Ensure the `Limits` are explicitly defined. If `Limits` are absent, the JVM will default to host-limits, consequently the CPU and memory settings will be calculated from there. See `unbouded deployments` for more details.

Leveraging `oc debug` and `inspect`

When an application behaves unexpectedly, you can use `oc debug` to inspect the runtime environment from inside the pod's namespace.

```
# Access the pod environment
oc debug pod/<pod-name> --image=registry.access.redhat.com/ubi9/ubi-minimal

# Once inside, inspect the cgroup limits directly
cat /sys/fs/cgroup/memory/memory.limit_in_bytes
```

Note: In cgroup v2 environments, the path will change:

cgroup Version	Memory Limit File Path
cgroups v1	`/sys/fs/cgroup/memory/memory.max_usage_in_bytes`
cgroups v2	`/sys/fs/cgroup/memory.max`

Using `jcmd`

The `jcmd` utility is the standard diagnostic tool for Java. Since Java 8u191+, the JVM is container-aware. You can use `jcmd` to verify how the JVM "sees" the container.

```
# Run jcmd inside the container to see VM flags - example with PID equals 1, given
example with hotrod-app
# oc exec <pod-name> -- jcmd $PID VM.info
oc exec <pod-name> -- jcmd 1 VM.info
```

- **Look for:** `Heap size`
- **Verification:** If `Heap size` (plus off-heap) is not adequate size compared to the container limit, your application is at risk of being killed by the kernel, the value must match the `MaxRAMPercentage`. The default `MaxRAMPercentage` value depends on how the application was started: run-java.sh sets (currently 80%), whereas 25% is the default value.

Troubleshooting a Resource-Constrained Pod

In this exercise, you will diagnose a pod experiencing memory constraints.

Prerequisites:

- An active OpenShift cluster.
- The `oc` CLI tool authenticated to your cluster.

Steps:

1. **Simulate a kill:**
    1. Deploy a pod with a very low memory limit (e.g., 64Mi) and a JVM heap requirement that

exceeds directly set via Xmx. Observe the kill:

```
oc get pods
# Look for OOMKilled status
oc get events
# Look at events happening
```

2. **Inspect the exit code:**

```
oc get pod <pod-name> -o
jsonpath='{.status.containerStatuses[0].state.terminated.reason}'
```

If the result is `OOMKilled`, the kernel terminated the process because it exceeded the cgroup memory limit.

1. **Verify the limits:** Check the deployment configuration to see the discrepancy between your `limit` and the JVM's Heap setting.
2. **Remediate and re-verify:** Adjust the deployment to either increase the container memory limit or lower the heap percentage via `JAVA_OPTS_APPEND="-XX:MaxRAMPercentage=50.0"` (available when `run-java.sh` is the entrypoint), then use `jcmd` inside the container to confirm the new `MaxHeapSize` is calculated correctly against the updated container limit.

```
env:
 - name: JAVA_OPTS_APPEND
     value: "-XX:MaxRAMPercentage=50.0"
```

The full tuning-and-verification walkthrough is covered in Analyzing Heap and Off-Heap Settings and summarized in Tuning Java Heap Settings Based on Container Limits.

Appendix

Complementary materials

Date	Title and Link
Mar 14, 2024	How to use Java container awareness in OpenShift 4
May 16, 2024	Using OpenShift 4's inspect for middleware troubleshooting
Jul 19, 2024	Metaspace setting and tuning in JDK 8+ Applications (in and outside containers)
Mar 05, 2025	How to troubleshoot Spring Boot in OpenShift
Nov 27, 2025	How does cgroups v2 impact Java, .NET, and Node.js in OpenShift 4?
Jun 18, 2026	Troubleshoot application misbehavior after an OpenShift upgrade

Frequently Asked Questions (FAQ) & Troubleshooting

Container Resource Limits and Cgroups

Q1: Should I run my applications without resource limits (unbounded)?

While it can be tempting to leave containers unbounded, you must weigh the performance potential against cluster stability.

Table 4. Pros and Cons of Unbounded Applications

Pros	Cons
• Use the full Host CPU and Memory • Can spike the full host capability	• Have BestEffort QoS • Competing with other applications

Q2: What is the impact of OpenShift migrating to cgroups v2?

OpenShift 4.19 entirely removes cgroups v1 support. This impacts how runtimes like the JVM and Node.js calculate their boundaries. For deep dives into these changes, refer to the following official resources:

- How does cgroups v2 impact Java and Node.js in OpenShift 4?
- Scenarios with Java under cgroups v2

Q3: How do the metric paths differ between cgroups v1 and v2?

cgroups v2 flattened and simplified the hierarchy. Here is where to find the equivalent memory metrics:

Table 5. cgroups v1 vs. v2 Memory Metrics

Metric	cgroups v1 Path	cgroups v2 Path
Peak Usage	`/sys/fs/cgroup/memory/memory.max_usage_in_bytes`	`/sys/fs/cgroup/memory.peak` (Requires Kernel 5.19+)
Current Usage	`/sys/fs/cgroup/memory/memory.usage_in_bytes`	`/sys/fs/cgroup/memory.current`
Hard Limit	`/sys/fs/cgroup/memory/memory.limit_in_bytes`	`/sys/fs/cgroup/memory.max`

Java Execution and Entry Points

Q4: How has JVM container memory awareness evolved?

Modern JVMs natively understand container limits, whereas previous non-container aware versions relied on the host’s physical RAM size.

Table 6. JVM Container Awareness Evolution

Feature	Previous Behavior (Pre-8u191)	Current Behavior (8u191+)
Memory Source	Queries `/proc/meminfo` (Host RAM)	Queries cgroups (Container Limit)
Heap Sizing	Requires manual `-Xmx` configuration	Automatically scales based on `-XX:MaxRAMPercentage`
Operational Impact	Decoupled; changing container size requires manual JVM tuning	Coupled; scaling container limits automatically adjusts heap

Note that cgroups v2 compliant is only enabled fully on Java 8u372, Java 11.0.16, Java 17+.

Q5: Should I use the `run-java.sh` script or start Java directly in my container?

Historically, `run-java.sh` was required to parse cgroups and pass the limits to the JVM. With modern Java versions, starting Java directly is often preferred.

Table 7. Execution Methods: run-java.sh vs. Direct Java Start

Feature	`run-java.sh` Entry Point	Straight `java` Start
Execution Method	Shell wrapper script that evaluates the environment and builds the Java command line.	Native binary execution directly invoking the JVM.
Environment Variables	Automatically parses custom variables like `JAVA_OPTS_APPEND` and `GC_COLLECTOR` (legacy variables like `JAVA_MAX_MEM_RATIO` are deprecated).	Ignores script-specific variables. Relies strictly on command-line arguments and standard environment variables like `JAVA_TOOL_OPTIONS`.
Default JVM Tuning	Automatically applies container-friendly defaults (e.g., `-XX:MaxRAMPercentage`, ParallelGC settings) if not overridden.	Uses standard JVM defaults unless explicitly defined in the command line or deployment manifest.

Feature	`run-java.sh` Entry Point	Straight `java` Start
Container Awareness	Historically required to calculate cgroups limits before Java had native container support.	Relies on native JVM container detection (standard in OpenJDK 8u191+, 11+, 17+, etc.) for cgroups v1 and v2.
Process Management	Typically uses `exec java` to ensure Java runs as PID 1, but adds initial shell evaluation overhead.	Directly executes as PID 1 with zero shell overhead, ideal for modern Kubernetes deployments.

For more specifics, check the Red Hat UBI OpenJDK Container Images Documentation.

Environment Variables

Q6: Which general environment variables are commonly found in these containers?

Table 8. Common Environment Variables

Environment Variable	Purpose
`PATH`	Specifies a list of directories containing executable programs.
`HOME`	Defines the absolute path to the current user's home directory.
`JAVA_HOME`	Points to the installation directory of the Java Runtime Environment (JRE) or Java Development Kit (JDK).
`KUBECONFIG`	Holds a list of paths to Kubernetes configuration files used by `kubectl` and other tools.

Q7: Which specific variables should I use to pass Java arguments?

Table 9. Java-Specific Environment Variables

Environment Variable	Purpose
`JAVA_OPTS`	Main option, which includes default settings - can be used to replace them
`JAVA_OPTS_APPEND`	Append option, can be used to complement the main ones
`GC_COLLECTOR`	Used for GC collector
`JAVA_TOOL_OPTIONS`	Should be avoided, it will be used for any Java environment variable

Q8: Why should `JAVA_TOOL_OPTIONS` be avoided?

Because it applies indiscriminately to **every** JVM invocation in that environment, including simple commands like `jcmd` or `jstat`, which can break them if the options are incompatible.

Example: Breaking sidecar tools with JAVA_TOOL_OPTIONS

```
$ export JAVA_TOOL_OPTIONS="APPLE"
$ jcmd
```

```
Picked up JAVA_TOOL_OPTIONS: APPLE
Unrecognized option: APPLE
Error: Could not create the Java Virtual Machine.
Error: A fatal exception has occurred. Program will exit.
```

Troubleshooting & Local Development

Q9: How do I interpret JVM diagnostic files like `VM.info`?

When diagnosing runtime behavior or crashes, refer to the official support documentation: Interpreting VM.info file in OpenJDK/OracleJDK

Q10: How can I build and test my OpenShift container images locally before deploying?

To replicate the OpenShift Source-to-Image (S2I) build process on your local machine, follow this guide: Build OpenJDK container images locally using standalone S2I tool

Resources

Download Course PDF